

Chu Văn Dũng
Số bài làm được: 32/32

MSV: 211210740

1. Định nghĩa hàm, truyền tham chiếu, giá trị của biến trong lời gọi hàm thay đổi sau khi thực hiện hàm.

Mô tả: Tạo một hàm thay đổi giá trị tham chiếu (cộng thêm 10 vào giá trị ban đầu).

Code:

```
fun modifyRef (x: int ref) =
  x := !x + 10; (* Thay đổi giá trị tham chiếu *)
val myRef = ref 5;
modifyRef myRef;
myRef;
```

Giao diện:

```
-
- fun modifyRef(x : int ref) =
= x := !x + 10;
val modifyRef = fn : int ref -> unit
- val myRef = ref 5
= ;
val myRef = ref 5 : int ref

- modifyRef myRef;
val it = () : unit
- myRef
= ;
val it = ref 15 : int ref
- myRef
= ;
val it = ref 15 : int ref
-
```

2. Định nghĩa 1 hàm đệ quy đuôi. Áp dụng đưa ra kết quả.

Mô tả: Tính Fibonacci đệ quy đuôi

Code:

```
fun fibonacci_tailrec(n, a, b) =
  if n = 0 then a
  else fibonacci_tailrec(n - 1, b, a + b);

fun fibonacci(n) = fibonacci_tailrec(n, 0, 1);
```

```
val result = fibonacci(10); (* Kết quả: 55 *)
```

Giao diện:

```
- (* Hàm tính Fibonacci sử dụng đệ quy đuôi *)
- fun fibonacci_tailrec(n, a, b) =
=   if n = 0 then a
=   else fibonacci_tailrec(n - 1, b, a + b);
val fibonacci_tailrec = fn : int * int * int -> int
-
- (* Hàm Fibonacci chính thức với tham số ban đầu *)
- fun fibonacci(n) = fibonacci_tailrec(n, 0, 1);
val fibonacci = fn : int -> int
-
- (* Gọi hàm để tính Fibonacci *)
- val result = fibonacci(10); (* Kết quả: 55 *)
val result = 55 : int
```

3. Định nghĩa hàm truyền hàm

Mô tả: Tạo một hàm applyTwice có thể được sử dụng trong các tình huống cần áp dụng một phép biến đổi nhiều lần lên một giá trị ban đầu, ví dụ:

- Nhân đôi một số hai lần

Code:

```
fun applyTwice(f, x) = f(f(x));
fun square(y) = y * y;
applyTwice(square, 2);
```

Giao diện:

```
- fun applyTwice(f, x) = f(f(x));
val applyTwice = fn : ('a -> 'a) * 'a -> 'a
- fun square(y) = y * y; (* Hàm tính bình phương *)
val square = fn : int -> int
- applyTwice(square, 2); (* Kết quả: square(square(2)) = square(4) = 16 *)
val it = 16 : int
-
```

4. Hàm trả về hàm

Mô tả: Tạo một hàm chuyển đổi. (Chuyển đổi từ kilogram sang pounds, chuyển đổi từ miles sang km). Hàm makeConverter nhận tham số đầu vào kiểu real. Trả về một hàm nhận tham số value (kiểu real) và trả về kết quả kiểu real.

Code:

```
fun makeConverter(factor:real) = fn value => value * factor;
val kgToPounds = makeConverter(2.20462); (* Chuyển đổi từ kg sang pounds *)
val milesToKm = makeConverter(1.60934); (* Chuyển đổi từ miles sang km *)
milesToKm(10.0);
kgToPounds(10.0);
```

Giao diện:

```
- fun makeConverter(factor:real) = fn value => value * factor;
val makeConverter = fn : real -> real -> real
- val kgToPounds = makeConverter(2.20462);
val kgToPounds = fn : real -> real
- kgToPounds(10.0);
val it = 22.0462 : real
- val milesToKm = makeConverter(1.60934);
val milesToKm = fn : real -> real
- milesToKm(10.0);
val it = 16.0934 : real
-
```

5. Insertion sort đa hình

Mô tả: Xây dựng hàm insertion sort với kiểu dữ liệu real.

Code:

```
fun less (a: real, b: real) = a < b;
fun insert(a, nil) = [a] | insert(a, b::t) = if less(a,b) then a::b::t else b::insert(a,t);
fun sort_ins nil = nil | sort_ins(a::t) = insert(a, sort_ins(t));
sort_ins([1.3,1.5,5.6,4.5,2.3,1.05]);
```

Giao diện:

```
- fun less (a: real, b: real) = a < b;
val less = fn : real * real -> bool
- fun insert(a, nil) = [a] | insert(a, b::t) = if less(a,b) then a::b::t else b::insert(a,t);
val insert = fn : real * real list -> real list
- fun sort_ins nil = nil | sort_ins(a::t) = insert(a, sort_ins(t));
val sort_ins = fn : real list -> real list
- sort_ins([1.3,1.5,5.6,4.5,2.3,1.05]);
val it = [1.05,1.3,1.5,2.3,4.5,5.6] : real list
-
```

6 Merge sort đa hình

Mô tả: Xây dựng hàm merge sort đa hình. (Dùng với kiểu dữ liệu string).

Ví dụ so sánh giữa 2 xâu:

```
less ("apple", "banana");(* true *)
less ("dog", "cattadfdsaf"); (* false *)
```

```
less ("hello", "helloo"); (* false *)
```

Code:

Hàm so sánh:

```
fun less (s1: string, s2: string) = s1 < s2;
```

Hàm merge:

```
fun merge ([], ys) = ys | merge (xs, []) = xs | merge (x::xs, y::ys) = if less(x,y) then
x :: merge(xs, y::ys)    else y :: merge(x::xs, ys);
```

Chia đôi danh sách:

```
fun splitList xs =
  let
    val mid = length xs div 2
    fun take (0, _) = []
      | take (_, []) = []
      | take (n, x::xs) = x :: take (n-1, xs)
```

```
    fun drop (0, xs) = xs
      | drop (_, []) = []
      | drop (n, _::xs) = drop (n-1, xs)
```

```
  in
    (take (mid, xs), drop (mid, xs))
  end;
```

Hàm mergesort:

```
fun mergesort [] = []
  | mergesort [x] = [x]
  | mergesort xs =
    let
      val (left, right) = splitList xs
    in
      merge (mergesort left, mergesort right)
    end;
```

Test case:

```
val stringList = ["chu van dung", "nguyen van nam", "messsi", "ronaldo", "dung",
"dat"];
mergesort(stringList);
```

Giao diện:

```

- fun less (s1: string, s2: string) = s1 < s2;
val less = fn : string * string -> bool

- fun merge ([], ys) = ys | merge (xs, []) = xs | merge (x::xs, y::ys) = if less(x,y) then x :: merge(xs, y::ys)
  else y :: merge(x::xs, ys);
val merge = fn : string list * string list -> string list

- fun splitList xs =
  = let
  =   val mid = length xs div 2
  =   fun take (0, _) = []
  =     | take (_, []) = []
  =     | take (n, x::xs) = x :: take (n-1, xs)
  =
  =   fun drop (0, xs) = xs
  =     | drop (_, []) = []
  =     | drop (n, _::xs) = drop (n-1, xs)
  = in
  =   (take (mid, xs), drop (mid, xs))
  = end;
val splitList = fn : 'a list -> 'a list * 'a list

- fun mergesort [] = []
  = | mergesort [x] = [x]
  = | mergesort xs =
  =   let
  =     val (left, right) = splitList xs
  =   in
  =     merge (mergesort left, mergesort right)
  =   end;
val mergesort = fn : string list -> string list

val stringList =
  ["chu van dung","nguyen van nam","messsi","ronaldo","dung","dat"]
  : string list
- mergesort(stringList);
val it = ["chu van dung","dat","dung","messsi","nguyen van nam","ronaldo"]
  : string list

```

7. Định nghĩa dữ liệu đệ quy

Mô tả: Định nghĩa danh sách liên kết.

Code:

datatype 'a LinkedList = Nil (* Danh sách rỗng *) | Cons of 'a * 'a LinkedList; (* Một phần tử + phần còn lại *)

val myList = Cons (1, Cons (2, Cons (3, Nil)));

Giao diện:

```

SML of New Jersey
Standard ML of New Jersey v110.83 [built: Thu May 31 09:04:19 2018]
- datatype 'a LinkedList = Nil (* Danh sách rỗng *)
= | Cons of 'a * 'a LinkedList; (* Một phần tử và phần còn lại *)
datatype 'a LinkedList = Cons of 'a * 'a LinkedList | Nil
- val myList = Cons (1, Cons (2, Cons (3, Nil)));
val myList = Cons (1,Cons (2,Cons #)) : int LinkedList
-

```

8. Map – reduce cho các phép toán khác cộng, nhân.

Mô tả: Tạo danh sách chữ cái đầu của mỗi từ (dùng toán tử $\wedge$ các chữ cái đầu của từng từ trong mảng đầu vào).

Code:

```

fun map(f,nil) = nil |
= map(f,x::xs) = f(x)::map(f,xs);
fun reduce (unit, opn, nil) = unit
= | reduce (unit, opn, h::t) = opn (h, reduce (unit, opn, t));
fun map_red(nil, f, unit, g) = unit | map_red(t, f, unit, g) = reduce(unit, g, map(f,t));
fun first_letter(s) = str (String.sub(s, 0));
map_red(["Chu", "Van", "Dung", "La Khê", "SML", "MAP", "reduce"], first_letter,
"", op ^);

```

Giao diện:

```

- fun map(f,nil) = nil |
= map(f,x::xs) = f(x)::map(f,xs);
val map = fn : ('a -> 'b) * 'a list -> 'b list
- fun reduce (unit, opn, nil) = unit
= | reduce (unit, opn, h::t) = opn (h, reduce (unit, opn, t));
val reduce = fn : 'a * ('b * 'a -> 'a) * 'b list -> 'a
- fun map_red(nil, f, unit, g) = unit | map_red(t, f, unit, g) = reduce(unit, g, map(f,t));
val map_red = fn : 'a list * ('a -> 'b) * 'c * ('b * 'c -> 'c) -> 'c

val first_letter = fn : string -> string
- map_red(["Chu", "Van", "Dung", "Study", "SML", "MAP", "reduce"], first_letter, "", op ^);
val it = "CVDSSMr" : string
-

```

9. Generic Programming trong C++

Mô tả: Tạo hàm findMax tìm giá trị lớn nhất trong một mảng, có thể áp dụng cho cả int, double, char...

Code:

```
#include <iostream>
```

```
template <typename T>
```

```
T findMax(T arr[], int size) {
```

```

    T maxVal = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > maxVal) {
            maxVal = arr[i];
        }
    }
    return maxVal;
}

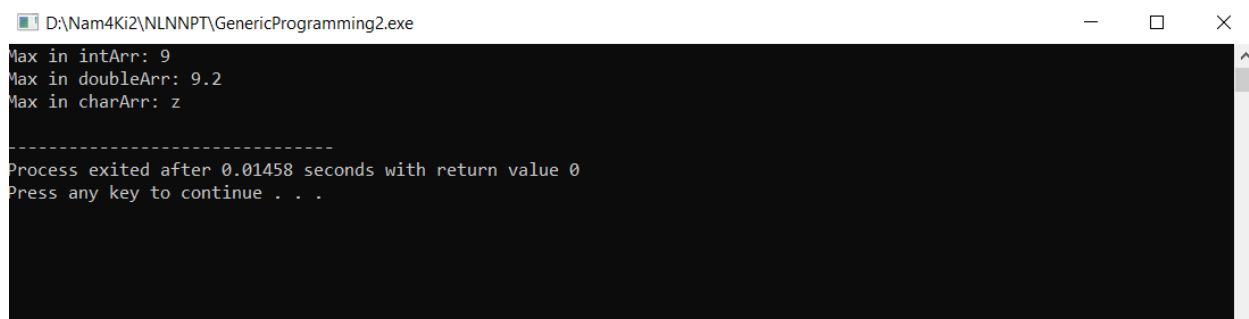
int main() {
    int intArr[] = {1, 5, 3, 7, 9};
    double doubleArr[] = {2.5, 3.7, 1.8, 9.2, 5.4};
    char charArr[] = {'a', 'z', 'b', 'm'};

    std::cout << "Max in intArr: " << findMax(intArr, 5) << std::endl;
    std::cout << "Max in doubleArr: " << findMax(doubleArr, 5) << std::endl;
    std::cout << "Max in charArr: " << findMax(charArr, 4) << std::endl;

    return 0;
}

```

Giao diện:



```

D:\Nam4Ki2\NLNNPT\GenericProgramming2.exe
Max in intArr: 9
Max in doubleArr: 9.2
Max in charArr: z

-----
Process exited after 0.01458 seconds with return value 0
Press any key to continue . . .

```

10.1 Lớp kế thừa Thread chạy

Mô tả: Mô phỏng quán cà phê bằng Thread

Chương trình có 2 loại luồng (số lượng 5):

- **Nhân viên pha chế** (Barista) sẽ pha cà phê ngẫu nhiên trong một khoảng thời gian.
- **Khách hàng** (Customer) sẽ chờ nhận cà phê và thưởng thức.

Code:

CoffeeShop:

```

class CoffeeShop {
    private String coffeeReady = null;

    public synchronized void makeCoffee(String coffee) {
        while (coffeeReady != null) {
            try {
                wait(); // Chờ khách hàng lấy cà phê trước khi pha cái mới
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        coffeeReady = coffee;
        System.out.println("☕ Nhân viên pha chế đã pha: " + coffee);
        notify(); // Thông báo cho khách hàng có cà phê
    }

    public synchronized void takeCoffee() {
        while (coffeeReady == null) {
            try {
                wait(); // Chờ cà phê được pha
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("Khách hàng nhận được: " + coffeeReady);
        coffeeReady = null;
        notify(); // Thông báo cho nhân viên pha chế có thể pha tiếp
    }
}

```

Barista:

```

class Barista extends Thread {
    private CoffeeShop shop;
    private String[] menu = {"Espresso", "Cappuccino", "Latte", "Mocha"};

    public Barista(CoffeeShop shop) {

```

```

        this.shop = shop;
    }

```

```

@Override
public void run() {
    for (int i = 0; i < 5; i++) {
        String coffee = menu[(int) (Math.random() * menu.length)];
        shop.makeCoffee(coffee);
        try {
            Thread.sleep((int) (Math.random() * 3000) + 1000); // Pha cà phê mất 1-3
giây
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```

Customer:

```

class Customer extends Thread {
    private CoffeeShop shop;

    public Customer(CoffeeShop shop) {
        this.shop = shop;
    }

    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            shop.takeCoffee();
            try {
                Thread.sleep((int) (Math.random() * 3000) + 1000); // Uống cà phê mất 1-
3 giây
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

```

    }
}

```

Main:

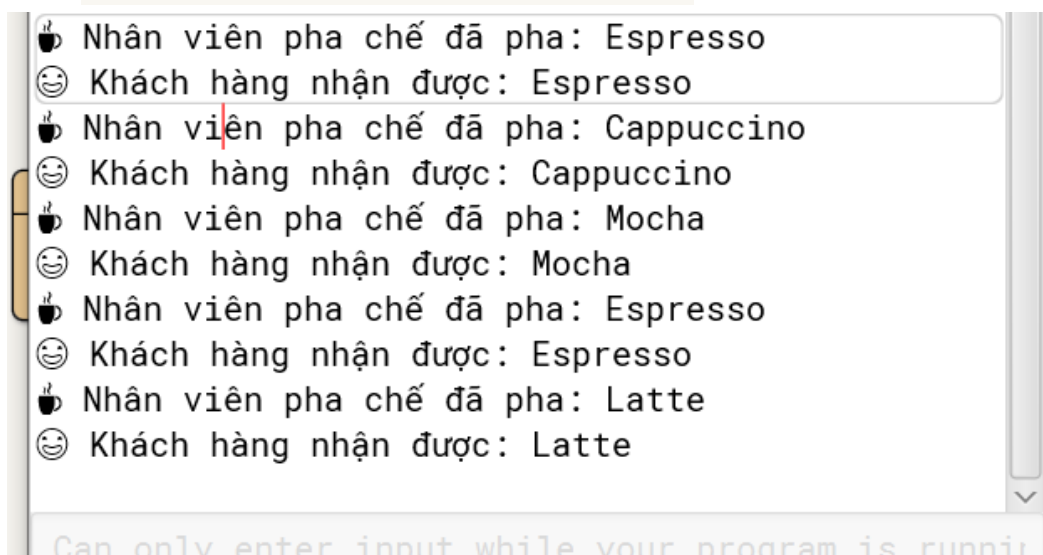
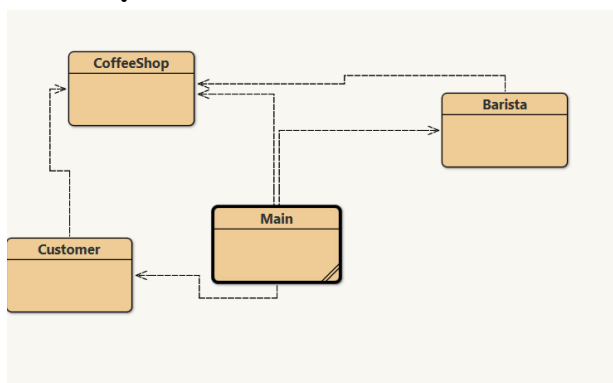
```

public class Main {
    public static void main(String[] args) {
        CoffeeShop shop = new CoffeeShop();

        Barista barista = new Barista(shop);
        Customer customer = new Customer(shop);

        barista.start();
        customer.start();
    }
}

```

Giao diện:

10.2. Lớp Implement Runnable chạy

Mô tả: **Mô phỏng quán cà phê bằng cách implement Runnable**

Chương trình có 2 loại luồng (số lượng 5):

- **Nhân viên pha chế** (Barista) sẽ pha cà phê ngẫu nhiên trong một khoảng thời gian.
- **Khách hàng** (Customer) sẽ chờ nhận cà phê và thưởng thức.

Code:

```
class CoffeeShop {
    private String coffeeReady = null;

    public synchronized void makeCoffee(String coffee) {
        while (coffeeReady != null) {
            try {
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        coffeeReady = coffee;
        System.out.println("☕ Nhân viên pha chế đã pha: " + coffee);
        notify();
    }

    public synchronized void takeCoffee() {
        while (coffeeReady == null) {
            try {
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("😊 Khách hàng nhận được: " + coffeeReady);
        coffeeReady = null;
        notify();
    }
}
```

```

class Barista implements Runnable {
    private CoffeeShop shop;

    public Barista(CoffeeShop shop) {
        this.shop = shop;
    }

    @Override
    public void run() {
        String[] menu = {"Espresso", "Latte", "Cappuccino", "Mocha", "Americano"};
        for (String coffee : menu) {
            shop.makeCoffee(coffee);
            try {
                Thread.sleep(1000); // Mô phỏng thời gian pha chế
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

class Customer implements Runnable {
    private CoffeeShop shop;

    public Customer(CoffeeShop shop) {
        this.shop = shop;
    }

    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            shop.takeCoffee();
            try {
                Thread.sleep(2000); // Mô phỏng thời gian uống cà phê
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

```

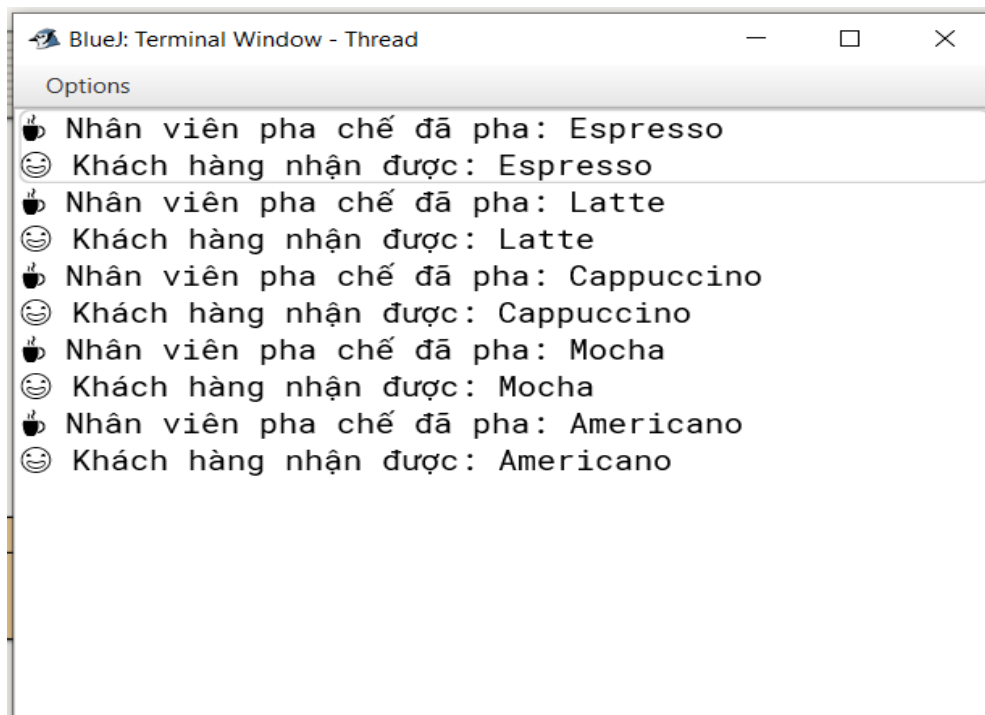
    }
}
}
public class Main {
    public static void main(String[] args) {
        CoffeeShop shop = new CoffeeShop();

        Thread baristaThread = new Thread(new Barista(shop)); // Tạo luồng nhân viên
pha chế
        Thread customerThread = new Thread(new Customer(shop)); // Tạo luồng
khách hàng

        baristaThread.start();
        customerThread.start();
    }
}

```

Giao diện:



11. Dùng 1 cấu trúc dữ liệu đa luồng

Sử dụng Cấu Trúc Dữ Liệu Đa Luồng (BlockingQueue) trong CoffeeShop

Trong môi trường đa luồng, nếu nhiều luồng **cùng đọc và ghi dữ liệu**, cần sử dụng **cấu trúc dữ liệu an toàn trong đa luồng** để tránh lỗi race condition.

Giải pháp: **BlockingQueue**

- **Tự động quản lý wait() và notify()** → Không cần dùng synchronized.
- **Thread-safe** → Có thể dùng nhiều luồng mà không bị lỗi race condition.
- **Hỗ trợ cơ chế chờ (take() và put())** giúp quản lý việc pha chế và nhận cà phê dễ dàng.

Code:

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

class CoffeeShop {
    private BlockingQueue<String> coffeeQueue = new LinkedBlockingQueue<>(1);
    // Chỉ chứa 1 ly cà phê mỗi lần

    public void makeCoffee(String coffee) {
        try {
            coffeeQueue.put(coffee); // Đặt cà phê vào hàng đợi (Chờ nếu đầy)
            System.out.println(" Nhân viên pha chế đã pha: " + coffee);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    public void takeCoffee() {
        try {
            String coffee = coffeeQueue.take(); // Lấy cà phê (Chờ nếu chưa có)
            System.out.println(" Khách hàng nhận được: " + coffee);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

class Barista implements Runnable {
    private CoffeeShop shop;
```

```

public Barista(CoffeeShop shop) {
    this.shop = shop;
}

@Override
public void run() {
    String[] menu = {"Espresso", "Latte", "Cappuccino", "Mocha", "Americano"};
    for (String coffee : menu) {
        shop.makeCoffee(coffee);
        try {
            Thread.sleep(1000); // Mô phỏng thời gian pha chế
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

class Customer implements Runnable {
    private CoffeeShop shop;

    public Customer(CoffeeShop shop) {
        this.shop = shop;
    }

    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            shop.takeCoffee();
            try {
                Thread.sleep(2000); // Mô phỏng thời gian uống cà phê
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

```

```

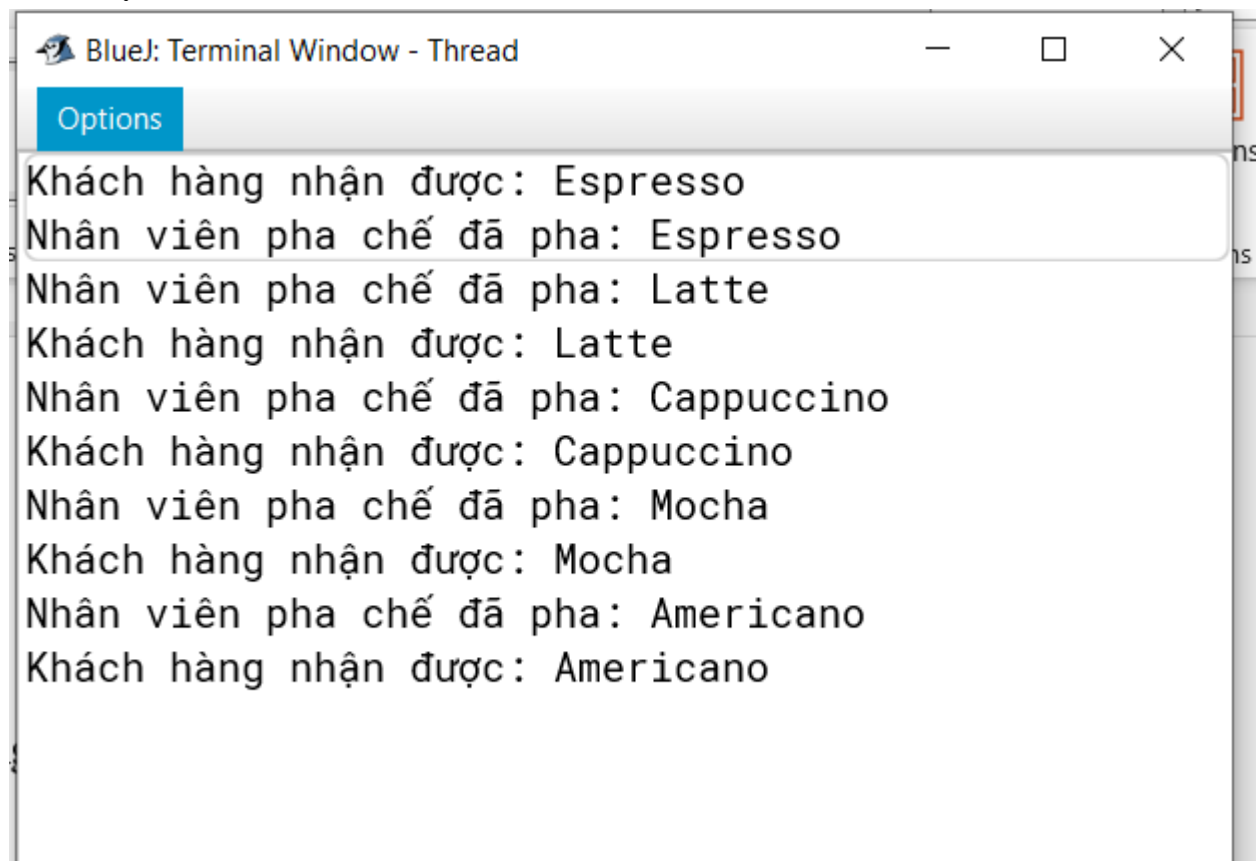
public class Main {
    public static void main(String[] args) {
        CoffeeShop shop = new CoffeeShop();

        Thread baristaThread = new Thread(new Barista(shop)); // Tạo luồng nhân viên pha chế
        Thread customerThread = new Thread(new Customer(shop)); // Tạo luồng khách hàng

        baristaThread.start();
        customerThread.start();
    }
}

```

Giao diện:



12. Chạy 1 chương trình lập lịch (chạy 1 chiến lược Java Thread trong O'Reilly Java Threads (2nd Edition))

Mô tả: Bộ lập lịch công việc tự động

Chương trình sử dụng **bộ lập lịch công việc (JobScheduler)** để thực thi các nhiệm vụ khác nhau tại thời điểm xác định hoặc theo chu kỳ lặp lại. Các công việc có thể bao gồm:

- Gửi báo cáo ngay lập tức hoặc sau một khoảng thời gian.
- Kiểm tra hệ thống sau một khoảng thời gian cụ thể.
- Sao lưu dữ liệu định kỳ.

Công việc	Thời điểm thực hiện	Lặp lại
Gửi báo cáo 1	Ngay lập tức	Không lặp
Gửi báo cáo 2	Sau 5 giây	Không lặp
Kiểm tra hệ thống	Sau 5 giây	Không lặp
Sao lưu dữ liệu	Mỗi 10 giây	Lặp 3 lần

Code:

JobScheduler:

```
import java.util.*;
```

```
public class JobScheduler implements Runnable {
    final public static int ONCE = 1;
    final public static int FOREVER = -1;
    final public static long HOURLY = (long)60*60*1000;
    final public static long DAILY = 24*HOURLY;
    final public static long WEEKLY = 7*DAILY;
    final public static long MONTHLY = -1;
    final public static long YEARLY = -2;
```

```
    private class JobNode {
        public Runnable job;
        public Date executeAt;
        public long interval;
        public int count;
    }
```

```
    private ThreadPool tp;
    private DaemonLock dlock = new DaemonLock();
```

```

private Vector jobs = new Vector(100);

public JobScheduler(int poolSize) {
    tp = (poolSize > 0) ? new ThreadPool(poolSize) : null;
    Thread js = new Thread(this);
    js.setDaemon(true);
    js.start();
}

private synchronized void addJob(JobNode job) {
    dlock.acquire();
    jobs.addElement(job);
    notify();
}

private synchronized void deleteJob(Runnable job) {
    for (int i=0; i < jobs.size(); i++) {
        if (((JobNode) jobs.elementAt(i)).job == job) {
            jobs.removeElementAt(i);
            dlock.release();
            break;
        }
    }
}

private JobNode updateJobNode(JobNode jn) {
    Calendar cal = Calendar.getInstance();
    cal.setTime(jn.executeAt);
    if (jn.interval == MONTHLY) {
        // There is a minor bug (see java.util.calendar).
        cal.add(Calendar.MONTH, 1);
        jn.executeAt = cal.getTime();
    } else if (jn.interval == YEARLY) {
        cal.add(Calendar.YEAR, 1);
        jn.executeAt = cal.getTime();
    } else {
        jn.executeAt =

```

```

        new Date(jn.executeAt.getTime() + jn.interval);
    }
    jn.count = (jn.count == FOREVER) ? FOREVER : jn.count - 1;
    return (jn.count != 0) ? jn : null;
}

```

```

private synchronized long runJobs() {
    long minDiff = Long.MAX_VALUE;
    long now = System.currentTimeMillis();

    for (int i=0; i < jobs.size(); i) {
        JobNode jn = (JobNode) jobs.elementAt(i);
        if (jn.executeAt.getTime() <= now) {
            if (tp != null) {
                tp.addRequest(jn.job);
            } else {
                Thread jt = new Thread(jn.job);
                jt.setDaemon(false);
                jt.start();
            }
            if (updateJobNode(jn) == null) {
                jobs.removeElementAt(i);
                dlock.release();
            }
        } else {
            long diff = jn.executeAt.getTime() - now;
            minDiff = Math.min(diff, minDiff);
            i++;
        }
    }
    return minDiff;
}

```

```

public synchronized void run() {
    while (true) {
        long waitTime = runJobs();
    }
}

```

```

        try {
            wait(waitTime);
        } catch (Exception e) {};
    }
}

public void execute(Runnable job) {
    executeIn(job, (long)0);
}

public void executeIn(Runnable job, long millis) {
    executeInAndRepeat(job, millis, 1000, ONCE);
}

public void executeInAndRepeat(Runnable job,
                                long millis, long repeat) {
    executeInAndRepeat(job, millis, repeat, FOREVER);
}

public void executeInAndRepeat(Runnable job, long millis,
                                long repeat, int count) {
    Date when = new Date(System.currentTimeMillis() + millis);
    executeAtAndRepeat(job, when, repeat, count);
}

public void executeAt(Runnable job, Date when) {
    executeAtAndRepeat(job, when, 1000, ONCE);
}

public void executeAtAndRepeat(Runnable job, Date when,
                                long repeat) {
    executeAtAndRepeat(job, when, repeat, FOREVER);
}

public void executeAtAndRepeat(Runnable job, Date when,
                                long repeat, int count) {
    JobNode jn = new JobNode();

```

```

    jn.job = job;
    jn.executeAt = when;
    jn.interval = repeat;
    jn.count = count;
    addJob(jn);
}

```

```

    public void cancel(Runnable job) {
        deleteJob(job);
    }
}

```

DaemonLock:

```

public class DaemonLock implements Runnable {
    private int lockCount = 0;

```

```

    public synchronized void acquire() {
        if (lockCount++ == 0) {
            Thread t = new Thread(this);
            t.setDaemon(false);
            t.start();
        }
    }
}

```

```

    public synchronized void release() {
        if (--lockCount == 0) {
            notify();
        }
    }
}

```

```

    public synchronized void run() {
        while (lockCount != 0) {
            try {
                wait();
            } catch (InterruptedException ex) {};
        }
    }
}

```

```
}
```

BusyFlag:

```
public class BusyFlag {
    protected Thread busyflag = null;
    protected int busycount = 0;
    public synchronized void
    getBusyFlag() {
        while (tryGetBusyFlag() == false) {
            try {
                wait();
            } catch (Exception e) {}
        }
    }
}
```

```
    public synchronized boolean tryGetBusyFlag() {
        if (busyflag == null) {
            busyflag = Thread.currentThread();
            busycount = 1;
            return true;
        }
        if (busyflag == Thread.currentThread()) {
            busycount++;
            return true;
        }
        return false;
    }
}
```

```
    public synchronized void freeBusyFlag() {

        if (getBusyFlagOwner() == Thread.currentThread()) {
            busycount--;
            if (busycount == 0) {
                busyflag = null;
                notify();
            }
        }
    }
}
```

```

    }

    public synchronized Thread getBusyFlagOwner() {
        return busyflag;
    }
}

```

ThreadPool:

```
import java.util.*;
```

```
public class ThreadPool {
```

```

    class ThreadPoolRequest {
        Runnable target;
        Object lock;
    }

```

```

        ThreadPoolRequest(Runnable t, Object l) {
            target = t;
            lock = l;
        }
    }

```

```

    class ThreadPoolThread extends Thread {
        ThreadPool parent;
        volatile boolean shouldRun = true;

        ThreadPoolThread(ThreadPool parent, int i) {
            super("ThreadPoolThread " + i);
            this.parent = parent;
        }
    }

```

```
    public void run() {
```

```

        ThreadPoolRequest obj = null;
        while (shouldRun) {
            try {
                parent.cvFlag.getBusyFlag();
            }
        }
    }
}

```

```

while (obj == null && shouldRun) {
    try {
        obj = (ThreadPoolRequest)
            parent.objects.elementAt(0);
        parent.objects.removeElementAt(0);
    } catch (ArrayIndexOutOfBoundsException aiobe) {
        obj = null;
    } catch (ClassCastException cce) {
        System.err.println("Unexpected data");
        obj = null;
    }
    if (obj == null) {
        try {
            parent.cvAvailable.cvWait();
        } catch (InterruptedException ie) {
            return;
        }
    }
} finally {
    parent.cvFlag.freeBusyFlag();
}
if (!shouldRun)
    return;
obj.target.run();
try {
    parent.cvFlag.getBusyFlag();
    nObjects--;
    if (nObjects == 0)
        parent.cvEmpty.cvSignal();
} finally {
    parent.cvFlag.freeBusyFlag();
}
if (obj.lock != null) {
    synchronized(obj.lock) {
        obj.lock.notify();
    }
}

```

```

        }
    }
    obj = null;
}
}

Vector objects;
int nObjects = 0;
CondVar cvAvailable, cvEmpty;
BusyFlag cvFlag;
ThreadPoolThread poolThreads[];
boolean terminated = false;

public ThreadPool(int n) {
    cvFlag = new BusyFlag();
    cvAvailable = new CondVar(cvFlag);
    cvEmpty = new CondVar(cvFlag);
    objects = new Vector();
    poolThreads = new ThreadPoolThread[n];
    for (int i = 0; i < n; i++) {
        poolThreads[i] = new ThreadPoolThread(this, i);
        poolThreads[i].start();
    }
}

private void add(Runnable target, Object lock) {

    try {
        cvFlag.getBusyFlag();
        if (terminated)
            throw new
                IllegalStateException("Thread pool has shut down");
        objects.addElement(new ThreadPoolRequest(target, lock));
        nObjects++;
        cvAvailable.cvSignal();
    } finally {

```

```

        cvFlag.freeBusyFlag();
    }
}

public void addRequest(Runnable target) {

    add(target, null);
}

public void addRequestAndWait(Runnable target)

    throws InterruptedException {
    Object lock = new Object();
    synchronized(lock) {
        add(target, lock);
        lock.wait();
    }
}

public void waitForAll(boolean terminate)
    throws InterruptedException {
    try {
        cvFlag.getBusyFlag();
        while (nObjects != 0)
            cvEmpty.cvWait();
        if (terminate) {
            for (int i = 0; i < poolThreads.length; i++)
                poolThreads[i].shouldRun = false;
            cvAvailable.cvBroadcast();
            terminated = true;
        }
    } finally {
        cvFlag.freeBusyFlag();
    }
}

```

```

    public void waitForAll() throws InterruptedException {
        waitForAll(false);
    }
}

CondVar:
public class CondVar {

    private BusyFlag SyncVar;

    public CondVar() {
        this(new BusyFlag());
    }

    public CondVar(BusyFlag sv) {
        SyncVar = sv;
    }

    public void cvWait() throws InterruptedException {
        cvTimedWait(SyncVar, 0);
    }

    public void cvWait(BusyFlag sv) throws InterruptedException {
        cvTimedWait(sv, 0);
    }

    public void cvTimedWait(int millis) throws InterruptedException {
        cvTimedWait(SyncVar, millis);
    }

    public void cvTimedWait(BusyFlag sv, int millis)
        throws InterruptedException {
        int i = 0;
        InterruptedException errex = null;

        synchronized (this) {
            // You must own the lock in order to use this method.

```

```

    if (sv.getBusyFlagOwner() != Thread.currentThread()) {
        throw new IllegalMonitorStateException(
            "current thread not owner");
    }

    // Release the lock (completely).
    while (sv.getBusyFlagOwner() == Thread.currentThread()) {
        i++;
        sv.freeBusyFlag();
    }

    // Use wait() method.
    try {
        if (millis == 0) {
            wait();
        } else {
            wait(millis);
        }
    } catch (InterruptedException iex) {
        errex = iex;
    }
}

// Obtain the lock (return to original state).
for (; i>0; i--) {
    sv.getBusyFlag();
}

if (errex != null) throw errex;
return;
}

public void cvSignal() {
    cvSignal(SyncVar);
}

public synchronized void cvSignal(BusyFlag sv) {

```

```

// You must own the lock in order to use this method.
if (sv.getBusyFlagOwner() != Thread.currentThread()) {
    throw new IllegalMonitorStateException(
        "current thread not owner");
}
notify();
}

public void cvBroadcast() {
    cvBroadcast(SyncVar);
}

public synchronized void cvBroadcast(BusyFlag sv) {
    // You must own the lock in order to use this method.
    if (sv.getBusyFlagOwner() != Thread.currentThread()) {
        throw new IllegalMonitorStateException(
            "current thread not owner");
    }
    notifyAll();
}
}

TestJobScheduler:
import java.util.Date;

public class TestJobScheduler {
    public static void main(String[] args) {
        JobScheduler scheduler = new JobScheduler(3);

        // Báo cáo 1: Gửi ngay lập tức
        Runnable reportJob1 = new Runnable() {
            @Override
            public void run() {
                System.out.println("Gửi báo cáo 1: " + new Date());
                System.out.println("Cập nhật dữ liệu cho báo cáo 1: " + new Date());
            }
        };
    }
};

```

```

scheduler.execute(reportJob1);

// Báo cáo 2: Gửi sau 5 giây
Runnable reportJob2 = new Runnable() {
    @Override
    public void run() {
        System.out.println("Gửi báo cáo 2: " + new Date());
        System.out.println("Cập nhật dữ liệu cho báo cáo 2: " + new Date());
    }
};
scheduler.executeIn(reportJob2, 5000);

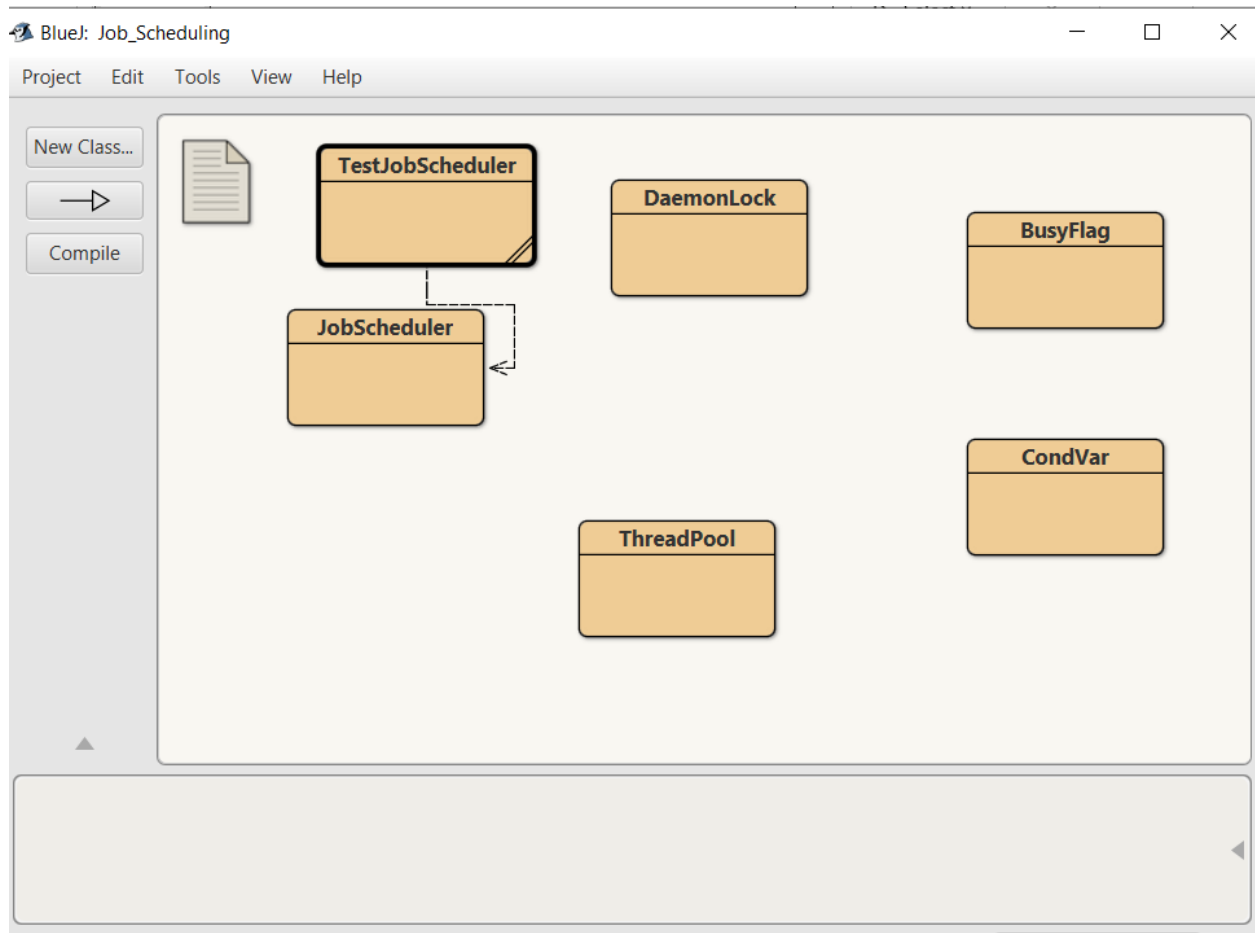
//Kiểm tra hệ thống sau 5 giây
Runnable healthCheckJob = new Runnable() {
    @Override
    public void run() {
        System.out.println("Kiểm tra hệ thống: " + new Date());
    }
};
scheduler.executeIn(healthCheckJob, 5000);

//Sao lưu dữ liệu mỗi 10 giây, lặp lại 3 lần
Runnable backupJob = new Runnable() {
    @Override
    public void run() {
        System.out.println("Đang sao lưu dữ liệu: " + new Date());
    }
};
scheduler.executeInAndRepeat(backupJob, 10000, 10000, 3);

// Chờ chương trình chạy 40 giây để kiểm tra kết quả
try {
    Thread.sleep(40000);
} catch (InterruptedException e) {
    e.printStackTrace();
}

```

```
}  
}  
Giao diện:
```



```
BlueJ: Terminal Window - Job_Scheduling
Options
Gửi báo cáo 1: Sat Mar 01 20:59:06 ICT 2025
Cập nhật dữ liệu cho báo cáo 1: Sat Mar 01 20:59:06 ICT 2025
Gửi báo cáo 2: Sat Mar 01 20:59:11 ICT 2025
Cập nhật dữ liệu cho báo cáo 2: Sat Mar 01 20:59:11 ICT 2025
Kiểm tra hệ thống: Sat Mar 01 20:59:11 ICT 2025
Đang sao lưu dữ liệu: Sat Mar 01 20:59:16 ICT 2025
Đang sao lưu dữ liệu: Sat Mar 01 20:59:26 ICT 2025
Đang sao lưu dữ liệu: Sat Mar 01 20:59:36 ICT 2025
Can only enter input while your program is running
```

13. Truyền hàm, trả về hàm (JavaScript)

Mô tả: Tạo một bộ lọc danh sách tùy chỉnh

Giả sử bạn có một danh sách số và muốn lọc các số theo tiêu chí nhất định (số chẵn, số lẻ, lớn hơn một giá trị nhất định, v.v). Tạo một **hàm tạo bộ lọc** giúp linh hoạt trong việc lọc danh sách.

Code:

```
function hamLoc(condition) {
  return function(numbers) {
```

```

        return numbers.filter(condition);
    };
}

const soChan = (num) => num % 2 === 0; // Số chẵn
const soLe = (num) => num % 2 !== 0; // Số lẻ
const lonHon5 = (num) => num > 5; // Lớn hơn 5

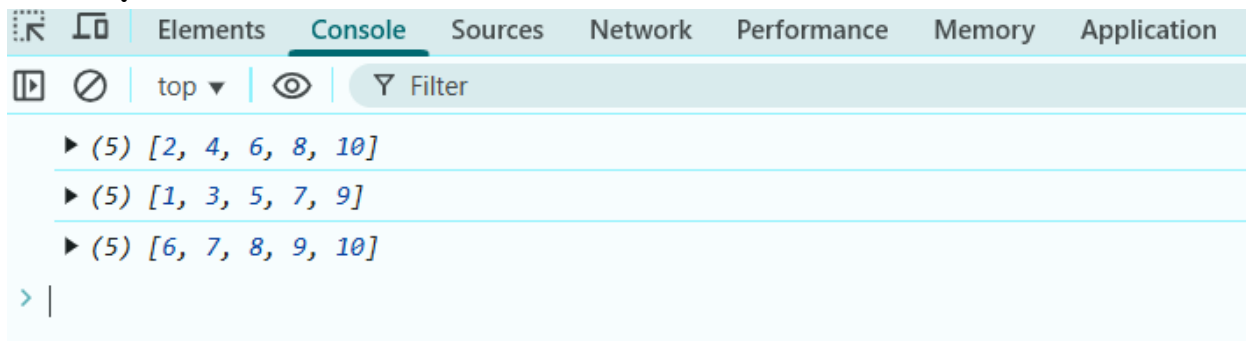
const locChan = hamLoc(soChan);
const locLe = hamLoc(soLe);
const locLonHon5 = hamLoc(lonHon5);

const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

console.log(locChan(numbers));
console.log(locLe(numbers));
console.log(locLonHon5(numbers));

```

Giao diện:



14. Dynamic Var, Prototype

Dynamic Var:

Mô tả: Giả sử ta có 2 đối tượng Car, ban đầu chỉ có các thuộc tính brand (hãng xe) và year (năm sản xuất). Tuy nhiên, ta muốn bổ sung thuộc tính color cho myCar1 sau khi khởi tạo đối tượng.

Code:

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Dynamic Prototype</title>
</head>
<body>
  <script>
    // Định nghĩa constructor function cho Car
    function Car(brand, year) {
      this.brand = brand;
      this.year = year;
    }

    // Tạo một đối tượng Car
    let myCar = new Car("Toyota", 2020);
    let myCar2 = new Car("Honda", 2003);
    console.log(myCar);
    // Output: { brand: 'Toyota', year: 2020 }

    // Thêm thuộc tính động (Dynamic Variable cho myCar)
    myCar.color = "Red";
    console.log(myCar);
    console.log(myCar2);

  </script>
</body>
</html>

```

Giao diện:



Prototype:

Mô tả: sử dụng **prototype** trong JavaScript để định nghĩa một phương thức lấy thông tin xe (getInfor) dùng chung cho tất cả các đối tượng Xe (Car) được tạo từ constructor function.

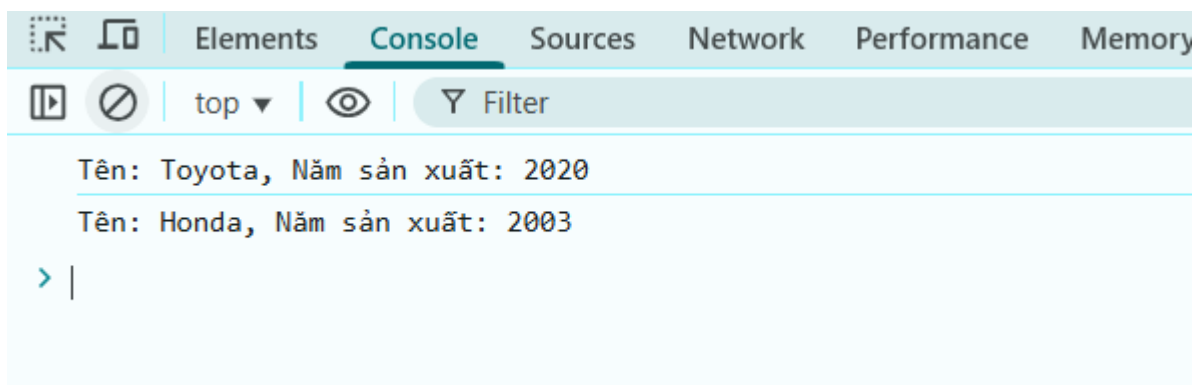
Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic Prototype</title>
</head>
<body>
  <script>
    // Định nghĩa constructor function cho Car
    function Car(brand, year) {
      this.brand = brand;
      this.year = year;
    }
    Car.prototype.getInfo = function() {
      return `Tên: ${this.brand}, Năm sản xuất: ${this.year}`;
    };
    // Tạo một đối tượng Car
    let myCar = new Car("Toyota", 2020);
    let myCar2 = new Car("Honda", 2003);

    console.log(myCar.getInfo());
    console.log(myCar2.getInfo());

  </script>
</body>
</html>
```

Giao diện:



15. MergeSort Prolog

Mô tả: Tạo hàm mergeSort trong Prolog

Code:

```
merge_sort([], []).
merge_sort([X], [X]).
```

```
merge_sort(List, Sorted) :-
    divide(List, Left, Right),
    merge_sort(Left, SortedLeft),
    merge_sort(Right, SortedRight),
    merge(SortedLeft, SortedRight, Sorted).
```

```
divide([], [], []).
divide([X], [X], []).
divide([X,Y|Rest], [X|Left], [Y|Right]) :-
    divide(Rest, Left, Right).
```

```
merge([], R, R).
merge(L, [], L).
merge([X|L], [Y|R], [X|M]) :- X <= Y, merge(L, [Y|R], M).
merge([X|L], [Y|R], [Y|M]) :- X > Y, merge([X|L], R, M).
```

Giao diện:

```
% d:/Nam4Ki2/NLNNPT/Prolog/mergersort.pl compiled 0.00 sec, 10 clauses
?-
|   merge_sort([10, 3, 7, 8, 2, 5], X).
X = [2, 3, 5, 7, 8, 10] .
?- ■
```

16. Tính toán trong prolog

Mô tả: Tính Fibonacci

Code:

```
fib_tail(N, Result) :- fib_helper(N, 0, 1, Result).
```

```
fib_helper(0, A, _, A).
```

```
fib_helper(N, A, B, Result) :-
```

```
    N > 0,
```

```
    N1 is N - 1,
```

```
    Next is A + B,
```

```
    fib_helper(N1, B, Next, Result).
```

Giao diện:

```
?-
% d:/Nam4Ki2/NLNNPT/Prolog/Fibonacci.pl compiled 0.02 sec, 3 clauses
?- fib_tail(92, X).
X = 7540113804746346429
```

17. Đảo xuôi_prolog

Mô tả:

1. Dùng một hàm đệ quy đuôi với tham số tích lũy (accumulator) để lưu kết quả tạm thời.
2. Lặp qua từng ký tự của chuỗi và đưa ký tự đó vào đầu danh sách tích lũy.
3. Khi hết chuỗi, danh sách tích lũy chính là chuỗi đảo ngược.

Code:

```
reverse_string(Input, Reversed) :-
```

```
    atom_chars(Input, Chars),      % Chuyển chuỗi thành kí tự
```

```
    reverse_tail_recursive(Chars, [], ReversedChars),
```

```
    atom_chars(Reversed, ReversedChars).
```

```
% De qui duoi
reverse_tail_recursive([], Acc, Acc).
reverse_tail_recursive([H|T], Acc, Reversed) :-
    reverse_tail_recursive(T, [H|Acc], Reversed) .
```

Giao diện:

```
warning:      previously defined at c:/users/cnu dung/appdata/local/temp/xpce4:/
% d:/Nam4Ki2/NLNNPT/Prolog/daoXau.pl compiled 0.00 sec, 3 clauses
% d:/nam4ki2/nlnnpt/prolog/daoXau compiled 0.00 sec, -2 clauses
?- reverse_string("hello", X).
X = olleh.

?- ■
```

18a. Ứng dụng nhỏ

Mô tả: bài toán **Quân Tượng (Bishops Problem)** - đặt N quân tượng trên bàn cờ N×N sao cho chúng không tấn công lẫn nhau, chỉ in ra **1 kết quả đầu tiên**

Code:

```
del(Item, [Item|List], List).
del(Item, [First|List], [First|List1]) :-
    del(Item, List, List1).

permutation([], []).
permutation([Head|Tail], PermList) :-
    permutation(Tail, PermTail),
    del(Head, PermList, PermTail).

%% Kiểm tra 2 quân tượng không tấn công nhau
%% (Tấn công khi cùng đường chéo: |X1-X2| = |Y1-Y2|)
bishops_not_attack((X1,Y1), (X2,Y2)) :-
    abs(X1 - X2) \= abs(Y1 - Y2).

safe_bishops([]).
safe_bishops([Bishop|Others]) :-
    safe_bishops(Others),
    maplist(bishops_not_attack(Bishop), Others).

solution_bishops(N, Bishops) :-
```

```
findall((X,Y), (between(1,N,X), between(1,N,Y)), AllPositions),
```

```
permutation(AllPositions, Bishops),
first_n(N, Bishops, Selected),
safe_bishops(Selected),
!.
```

```
first_n(0, _, []).
first_n(N, [H|T], [H|Rest]) :-
    N > 0,
    N1 is N - 1,
    first_n(N1, T, Rest).
```

Giao diện:

```
solution_bishops(8, Bishops), writeln(Bishops).
```

```
[1] ?- solution_bishops(8, Bishops), writeln(Bishops).
[(1,1),(1,2),(1,3),(1,4),(1,5),(1,6),(1,7),(1,8),(2,1),(2,2),(2,3),(2,4),(2,5),(2,6),(2,7),(2,8),(3,1),(3,2),(3,3),(3,4),(3,5),(3,6),(3,7),(3,8),(4,1),(4,2),(4,3),(4,4),(4,5),(4,6),(4,7),(4,8),(5,1),(5,2),(5,3),(5,4),(5,5),(5,6),(5,7),(5,8),(6,1),(6,2),(6,3),(6,4),(6,5),(6,6),(6,7),(6,8),(7,1),(7,2),(7,3),(7,4),(7,5),(7,6),(7,7),(7,8),(8,1),(8,2),(8,3),(8,4),(8,5),(8,6),(8,7),(8,8)]
Bishops = [(1,1),(1,2),(1,3),(1,4),(1,5),(1,6),(1,7),(1,8),(....)....]
[1] ?- ■
```

18b Mô tả: Ứng dụng map: Áp dụng map để xử lý danh sách các hàm khác nhau, mỗi hàm sẽ được áp dụng lên cùng một giá trị.

Code:

```
% apply_functions(ListOfFunctions, Input, ListOfResults)
apply_functions([], _, []).
apply_functions([Func|Funcs], X, [Y|Ys]) :-
    call(Func, X, Y),
    apply_functions(Funcs, X, Ys).
```

```
% Hàm tính toán khác nhau
```

```
square(N, R) :- R is N * N.
```

```
double(N, R) :- R is N * 2.
```

```
increment(N, R) :- R is N + 1.
```

```
% Truy vấn:
```

```
apply_functions([square, double, increment], 3, Result).
```

Giao diện:

```
% d:/Nam4Ki2/NLNNPT/Prolog/map.pl compiled 0.00 sec, 5 clauses
?- apply_functions([square, double, increment], 3, Result).
Result = [9, 6, 4].

?- ■
```

19. Tính gần đúng sinx

Mô tả:

Sử dụng chuỗi Taylor để tính gần đúng giá trị của $\sin(x)$ với sai số ϵ . Chuỗi Taylor của $\sin(x)$ tại $x=0$ là:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} \dots$$

Code:

```
import math
```

```
def next_term(x, n, term):
```

```
    return term * (-x**2 / ((2*n) * (2*n + 1)))
```

```
def sin_taylor(x, epsilon=0.01):
```

```
    def iter_terms():
```

```
        """Generator sinh từng hạng tử trong chuỗi Taylor"""
```

```
        n = 1
```

```
        term = x # Hạng tử đầu tiên là x
```

```
        while True:
```

```
            yield term
```

```
            term = next_term(x, n, term) # Tính hạng tử tiếp theo
```

```
            n += 1
```

```
def within(epsilon, iterable):
```

```
    """Tìm giá trị hội tụ của chuỗi khi sai số giữa 2 lần lặp < epsilon"""
```

```
    total = 0
```

```
    prev = 0
```

```
    for term in iterable:
```

```
        total += term
```

```
        if abs(total - prev) <= epsilon:
```

```
            return total
```

```
prev = total # Cập nhật giá trị trước đó
```

```
return within(epsilon, iter_terms()) # Bắt đầu tính chuỗi
```

```
x = math.radians(30)
```

```
epsilon = 0.0001
```

```
approx_value = sin_taylor(x, epsilon)
```

```
true_value = math.sin(x)
```

```
print(f"Giá trị gần đúng của sin(30°): {approx_value}")
```

```
print(f"Giá trị thực của sin(30°): {true_value}")
```

Giao diện:

```
term = x # Hạng tử đầu tiên là x
while True:
    yield term
    term = next_term(x, n, term) # Tính hạng tử tiếp theo
    n += 1

def within(epsilon, iterable):
    """Tìm giá trị hội tụ của chuỗi khi sai số giữa 2 lần lặp < epsilon"""
    total = 0
    prev = 0
    for term in iterable:
        total += term
        if abs(total - prev) <= epsilon:
            return total
        prev = total # Cập nhật giá trị trước đó
    return within(epsilon, iter_terms()) # Bắt đầu tính chuỗi

x = math.radians(30)
epsilon = 0.0001
approx_value = sin_taylor(x, epsilon)
true_value = math.sin(x)
print(f"Giá trị gần đúng của sin(30°): {approx_value}")
print(f"Giá trị thực của sin(30°): {true_value}")

Giá trị gần đúng của sin(30°): 0.4999999918690232
Giá trị thực của sin(30°): 0.49999999999999994
```

20. Dùng map vào bộ duyệt

Mô tả: Dùng map() để tính các giá trị sin(x) từ danh sách góc

Code:

```
import math
```

```
angles = [0, 30, 45, 60, 90] # Danh sách góc (độ)
```

```
radians = map(math.radians, angles) # Chuyển đổi độ sang radian
```

```
sin_values = map(math.sin, radians) # Tính sin của từng góc
```

```
print(f'{"Góc".ljust(6)} | {"sin(Góc)".rjust(10)}')
```

```
print("-" * 20)
```

```
for angle, value in zip(angles, sin_values):
    print(f"{str(angle).ljust(6)} | {value:10.6f}")
```

Giao diện:

The screenshot shows a Jupyter Notebook cell with the following code:

```
[25]: import math

angles = [0, 30, 45, 60, 90] # Danh sách góc (độ)
radians = map(math.radians, angles) # Chuyển đổi độ sang radian
sin_values = map(math.sin, radians) # Tính sin của từng góc

print(f"Góc'.ljust(6)} | {'sin(Góc)'.rjust(10)}")
print("-" * 20)

for angle, value in zip(angles, sin_values):
    print(f"{str(angle).ljust(6)} | {value:10.6f}")
```

The output of the code is a table with two columns: 'Góc' and 'sin(Góc)'. The table contains five rows of data for angles 0, 30, 45, 60, and 90 degrees.

Góc	sin(Góc)
0	0.000000
30	0.500000
45	0.707107
60	0.866025
90	1.000000

21. Dùng callable vào object để cài đặt một hàm theo mẫu strategy.

Mô tả: Chuyển đổi số tiền từ các loại tiền tệ khác nhau sang VND (Việt Nam Đồng). (usd, eur, cyn).

Code:

```
class CurrencyConverter:
```

```
    def __init__(self, conversion_strategy):
        self.conversion_strategy = conversion_strategy
```

```
    def __call__(self, amount):
        return self.conversion_strategy(amount)
```

```
def usd_to_vnd(amount):
    return amount * 25594
```

```
def eur_to_vnd(amount):
    return amount * 26754
```

```
def Chinese_Yuan_to_vnd(amount):
    return amount * 3513
```

Sử dụng

```
convert_usd = CurrencyConverter(usd_to_vnd)
```

```
convert_eur = CurrencyConverter(eur_to_vnd)
```

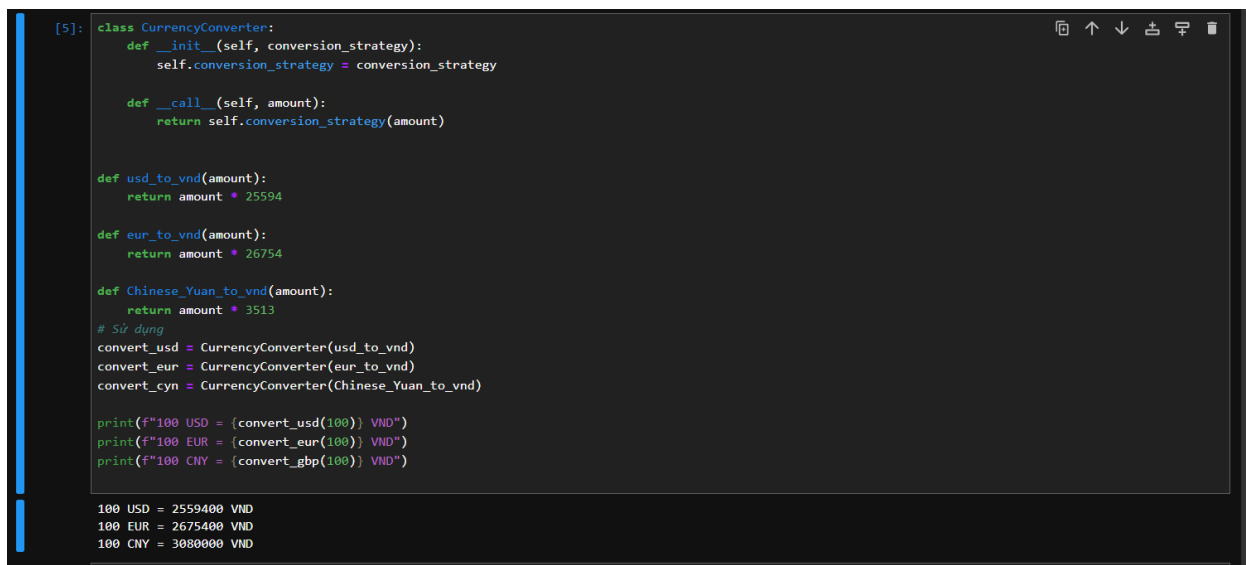
```
convert_cyn = CurrencyConverter(Chinese_Yuan_to_vnd)
```

```
print(f"100 USD = {convert_usd(100)} VND")
```

```
print(f"100 EUR = {convert_eur(100)} VND")
```

```
print(f"100 CNY = {convert_gbp(100)} VND")
```

Giao diện:



```
[5]: class CurrencyConverter:
    def __init__(self, conversion_strategy):
        self.conversion_strategy = conversion_strategy

    def __call__(self, amount):
        return self.conversion_strategy(amount)

    def usd_to_vnd(amount):
        return amount * 25594

    def eur_to_vnd(amount):
        return amount * 26754

    def Chinese_Yuan_to_vnd(amount):
        return amount * 3513

# Sử dụng
convert_usd = CurrencyConverter(usd_to_vnd)
convert_eur = CurrencyConverter(eur_to_vnd)
convert_cyn = CurrencyConverter(Chinese_Yuan_to_vnd)

print(f"100 USD = {convert_usd(100)} VND")
print(f"100 EUR = {convert_eur(100)} VND")
print(f"100 CNY = {convert_gbp(100)} VND")

100 USD = 2559400 VND
100 EUR = 2675400 VND
100 CNY = 3513000 VND
```

22. Lập hàm tính phi hàm Euler

Mô tả:

Nếu:

$$n = p_1^{k_1} \times p_2^{k_2} \times \dots \times p_m^{k_m}$$

thì:

$$\phi(n) = (p_1^{k_1} - p_1^{k_1-1}) \times (p_2^{k_2} - p_2^{k_2-1}) \times \dots \times (p_m^{k_m} - p_m^{k_m-1})$$

Code:

```
def phi_euler(n):
```

```
    result = 1 # Khởi tạo giá trị kết quả
```

```
    p = 2 # Bắt đầu từ số nguyên tố đầu tiên
```

```
    while p * p <= n: # Lặp qua tất cả các số nguyên tố ≤ sqrt(n)
```

```
        if n % p == 0:
```

```
            k = 0
```

```
            while n % p == 0:
```

```
                n //= p
```

```

        k += 1 # Đếm số mũ k của p trong n
        result *= (p ** k - p ** (k - 1)) # Áp dụng công thức  $p^k - p^{(k-1)}$ 
    p += 1

```

```

if n > 1: # Nếu còn lại một số nguyên tố lớn hơn 1
    result *= (n - 1)

```

```

return result

```

```

# Kiểm tra

```

```

print(phi_euler(10))
print(phi_euler(100))
print(phi_euler(81))
print(phi_euler(13))

```

Giao diện:

```

[9]: def phi_euler(n):
    result = 1 # Khởi tạo giá trị kết quả
    p = 2 # Bắt đầu từ số nguyên tố đầu tiên

    while p * p <= n: # Lặp qua tất cả các số nguyên tố ≤ sqrt(n)
        if n % p == 0:
            k = 0
            while n % p == 0:
                n //= p
                k += 1 # Đếm số mũ k của p trong n
            result *= (p ** k - p ** (k - 1)) # Áp dụng công thức  $p^k - p^{(k-1)}$ 
            p += 1

    if n > 1: # Nếu còn lại một số nguyên tố lớn hơn 1
        result *= (n - 1)

    return result

# Kiểm tra
print(phi_euler(10))
print(phi_euler(100))
print(phi_euler(81))
print(phi_euler(13))

```

4
40
54
12

23. Tính mean(z). Đóng bộ 3.

Mô tả: Đọc file Anscombe.csv với 3 biến x,y,z cho từng quý. Tính mean(z). Đóng bộ 3 cho từng quý.

Dữ liệu file:

Anscombe.csv - Notepad

File Edit Format View Help

Anscombe's quartet

I	II	III	IV								
x	y	z	x	y	z	x	y	z	x	y	z
10.0	8.04	21	10.0	9.14	23.4	10.0	7.46	3.4	8.0	6.58	7.7
8.0	6.95	8.0	8.14	8.0	6.77	8.0	5.76	21	5	67	8
13.0	7.58	13.0	8.74	13.0	12.74	8.0	7.71	7	9	0	-2.2

Code:

```
import csv
def row_iter(source):
    return csv.reader(source, delimiter="\t")
with open("Anscombe.csv") as source:
    print( list(row_iter(source)) )
def head_split_fixed(row_iter):
    title= next(row_iter)
    assert len(title) == 1 and title[0] == "Anscombe's quartet"
    heading= next(row_iter)
    assert len(heading) == 4 and heading == ['I', 'II', 'III', 'IV']
    columns= next(row_iter)
    assert len(columns) == 12 and columns == ['x', 'y','z', 'x', 'y','z', 'x',
        'y','z', 'x', 'y','z']
    return row_iter
with open("Anscombe.csv") as source:
    print( list(head_split_fixed(row_iter(source))))
from collections import namedtuple
Trip = namedtuple("Trip", ("x", "y", "z"))
def series(n, row_iter):
    for row in row_iter:
        yield Trip(*row[n*3:n*3+3])
with open("Anscombe.csv") as source:
    data = tuple(head_split_fixed(row_iter(source)))
    sample_I= tuple(series(0,data))
    sample_II= tuple(series(1,data))
    sample_III= tuple(series(2,data))
    sample_IV= tuple(series(3,data))
mean = sum(float(trip.z) for trip in sample_I)/len(sample_I)
quy = 1
```

```

new_trips=[]
for subset in sample_I, sample_II, sample_III, sample_IV:

    mean = sum(float(trip.z) for trip in subset)/len(subset)
    new_trips.append([(trip.x, trip.y, mean) for trip in subset])
    print(f"Mean(z) quý {quy}: {mean}")
    quy+=1
new_trip_indx = 1
for new_trip in new_trips:

    print(f"Quy {new_trip_indx}: {new_trip}")
    new_trip_indx+=1

```

Giao diện:

```

[7]: import csv
def row_iter(source):
    return csv.reader(source, delimiter="\t")

[8]: with open("Anscombe.csv") as source:
    print( list(row_iter(source)) )

[["Anscombe's quartet"], ['I', 'II', 'III', 'IV'], ['x', 'y', 'x', 'y', 'x', 'y', 'x', 'y'], ['10.0', '8.04', '10.0', '9.14', '10.0', '7.46', '8.0', '6.58'], ['8.0', '6.95', '8.0', '8.14', '8.0', '6.77', '8.0', '5.76'], ['13.0', '7.58', '13.0', '8.74', '13.0', '12.74', '8.0', '7.71']]

[21]: def head_split_fixed(row_iter):
    title= next(row_iter)
    assert len(title) == 1 and title[0] == "Anscombe's quartet"
    heading= next(row_iter)
    assert len(heading) == 4 and heading == ['I', 'II', 'III', 'IV']
    columns= next(row_iter)
    assert len(columns) == 12 and columns == ['x', 'y', 'z', 'x', 'y', 'z', 'x', 'y', 'z', 'x', 'y', 'z']
    return row_iter

[22]: with open("Anscombe.csv") as source:
    print( list(head_split_fixed(row_iter(source))))

[['10.0', '8.04', '21', '10.0', '9.14', '23.4', '10.0', '7.46', '3.4', '8.0', '6.58', '7.7'], ['8.0', '6.95', '8.0', '8.14', '8.0', '6.77', '8.0', '5.76', '21', '5', '67', '8'], ['13.0', '7.58', '13.0', '8.74', '13.0', '12.74', '8.0', '7.71', '7', '9', '0', '-2.2']]

[56]: quy = 1
new_trips=[]
for subset in sample_I, sample_II, sample_III, sample_IV:

    mean = sum(float(trip.z) for trip in subset)/len(subset)
    new_trips.append([(trip.x, trip.y, mean) for trip in subset])
    print(f"Mean(z) quý {quy}: {mean}")
    quy+=1

Mean(z) quý 1: 14.0
Mean(z) quý 2: 14.303333333333333
Mean(z) quý 3: 10.466666666666667
Mean(z) quý 4: 4.5

Đóng bộ 3 cho từng quý

[46]: new_trip_indx = 1
for new_trip in new_trips:

    print(f"Quy {new_trip_indx}: {new_trip}")
    new_trip_indx+=1

Quy 1: [['10.0', '8.04', 14.0], ['8.0', '6.95', 14.0], ['13.0', '7.58', 14.0]]
Quy 2: [['10.0', '9.14', 14.303333333333333], ['8.14', '8.0', 14.303333333333333], ['8.74', '13.0', 14.303333333333333]]
Quy 3: [['10.0', '7.46', 10.466666666666667], ['8.0', '5.76', 10.466666666666667], ['8.0', '7.71', 10.466666666666667]]
Quy 4: [['8.0', '6.58', 4.5], ['5', '67', 4.5], ['9', '0', 4.5]]

```

24. Đọc từ file color.csv.

Mô tả: Đọc từ file color.csv. Đưa dữ liệu vào sequence colors. Rồi đưa vào từ điển, in và get

Dữ liệu file:



corlor.csv - Notepad

File Edit Format View Help

GIMP Palette

Name: Small

Columns: 3

#

0	0	0	Black
255	255	255	White
238	32	77	Red
28	172	120	Green
31	117	254	Blue

Code:

```
import csv
def row_iter(source):
    return csv.reader(source, delimiter="\t")
with open("corlor.csv") as source:
    print( list(row_iter(source)) )
def head_split_fixed(row_iter):
    title= next(row_iter)
    assert len(title) == 1 and title[0] == "GIMP Palette"
    heading= next(row_iter)
    assert len(heading) == 1 and heading == ['Name: Small']
    columns= next(row_iter)
    assert len(columns) == 1 and columns == ['Columns: 3']
    columns= next(row_iter)
    assert len(columns) == 1 and columns == ['#']
    return row_iter
with open("corlor.csv") as source:
```

```

    print( list(head_split_fixed(row_iter(source))))
from collections import namedtuple
Color = namedtuple("Color", ("red", "green", "blue", "name"))
sequence = []
with open("corlor.csv") as source:
    row_iter = tuple(head_split_fixed(row_iter(source)))
    for row in row_iter:
        sequence.append(Color(*row))
sequence
name_map = dict((c.name, c) for c in sequence)
name_map['Black']
print("Apricot" in name_map)
print("Blue" in name_map)
name_map.get("Apricot")
name_map.get("Blue")

```

Giao diện:

```

24

[35]: import csv
      def row_iter(source):
          return csv.reader(source, delimiter="\t")

[36]: with open("corlor.csv") as source:
      print( list(row_iter(source)) )

[['GIMP Palette'], ['Name: Small'], ['Columns: 3'], ['#'], ['0', '0', '0', 'Black'], ['255', '255', '255', 'White'], ['238', '32', '77', 'Red'], ['28',
'172', '120', 'Green'], ['31', '117', '254', 'Blue']]

[30]: def head_split_fixed(row_iter):
      title= next(row_iter)
      assert len(title) == 1 and title[0] == "GIMP Palette"
      heading= next(row_iter)
      assert len(heading) == 1 and heading == ['Name: Small']
      columns= next(row_iter)
      assert len(columns) == 1 and columns == ['Columns: 3']
      columns= next(row_iter)
      assert len(columns) == 1 and columns == ['#']
      return row_iter

[31]: with open("corlor.csv") as source:
      print( list(head_split_fixed(row_iter(source))))

[['0', '0', '0', 'Black'], ['255', '255', '255', 'White'], ['238', '32', '77', 'Red'], ['28', '172', '120', 'Green'], ['31', '117', '254', 'Blue']]

[37]: from collections import namedtuple
      Color = namedtuple("Color", ("red", "green", "blue", "name"))
      sequence = []
      with open("corlor.csv") as source:
          row_iter = tuple(head_split_fixed(row_iter(source)))
          for row in row_iter:
              sequence.append(Color(*row))

[38]: sequence

[38]: [Color(red='0', green='0', blue='0', name='Black'),
      Color(red='255', green='255', blue='255', name='White'),
      Color(red='238', green='32', blue='77', name='Red'),
      Color(red='28', green='172', blue='120', name='Green'),
      Color(red='31', green='117', blue='254', name='Blue')]

[39]: name_map = dict((c.name, c) for c in sequence)

[40]: name_map['Black']

[40]: Color(red='0', green='0', blue='0', name='Black')

```

```
[40]: Color(red='0', green='0', blue='0', name='Black')

[44]: print("Apricot" in name_map)
      print("Blue" in name_map)

False
True

[42]: name_map.get("Apricot")

[43]: name_map.get("Blue")

[43]: Color(red='31', green='117', blue='254', name='Blue')

[ ]:
```

25 Đọc file map.txt

Mô tả: Đọc file map.txt gồm các tọa độ (x, y). Đóng bộ ba từng điểm. Tính chu vi ba điểm → Tạo bộ 4 (p1, p2, p3, chuvi)

Code:

```
def read_points(filename):
    with open(filename) as f:
        for line in f:
            yield tuple(map(float, line.split()))
points = read_points("map.txt")
list(points)
def triplets(iterable):
    def triplet_from(first, second, iterable_tail):
        try:
            third = next(iterable_tail)
            yield first, second, third
            yield from triplet_from(second, third, iterable_tail)
        except StopIteration:
            return

    try:
        iterator = iter(iterable)
        return triplet_from(next(iterator), next(iterator), iterator) #
    except StopIteration:
        return

trips_iter = triplets(read_points("map.txt"))
for trip in trips_iter:
    print(trip)
import math
```

```

def chuvi(point1, point2, point3):
    def distance(p1, p2):
        return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)

    return distance(point1, point2) + distance(point2, point3) + distance(point3,
point1)
trips_iter = triplets(read_points("map.txt"))
quad = ((triplet[0], triplet[1], triplet[2], chuvi(*triplet)) for triplet in trips_iter)
for quad_iter in quad:
    print(quad_iter)

```

Giao diện:

```

[57]: def read_points(filename):
        with open(filename) as f:
            for line in f:
                yield tuple(map(float, line.split()))

[58]: points = read_points("map.txt")

[59]: list(points)

[59]: [(1.0, 2.0), (3.0, 4.0), (5.0, 6.0), (7.0, 8.0), (9.0, 10.0), (11.0, 12.0)]

[60]: list(points)

[60]: []

```

```
[61]: def triplets(iterable):
        def triplet_from(first, second, iterable_tail):
            try:
                third = next(iterable_tail)
                yield first, second, third
                yield from triplet_from(second, third, iterable_tail)
            except StopIteration:
                return

        try:
            iterator = iter(iterable)
            return triplet_from(next(iterator), next(iterator), iterator) #
        except StopIteration:
            return
```

```
[65]: trips_iter = triplets(read_points("map.txt"))
```

```
[66]: for trip in trips_iter:
        print(trip)

((1.0, 2.0), (3.0, 4.0), (5.0, 6.0))
((3.0, 4.0), (5.0, 6.0), (7.0, 8.0))
((5.0, 6.0), (7.0, 8.0), (9.0, 10.0))
((7.0, 8.0), (9.0, 10.0), (11.0, 12.0))
```

```
[73]: import math

        def chuvi(point1, point2, point3):
            def distance(p1, p2):
                return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)

            return distance(point1, point2) + distance(point2, point3) + distance(point3, point1)
```

```
[74]: trips_iter = triplets(read_points("map.txt"))
        quad = ((triplet[0], triplet[1], triplet[2], chuvi(*triplet)) for triplet in trips_iter)
```

```
[75]: for quad_iter in quad:
        print(quad_iter)

((1.0, 2.0), (3.0, 4.0), (5.0, 6.0), 11.313708498984761)
((3.0, 4.0), (5.0, 6.0), (7.0, 8.0), 11.313708498984761)
((5.0, 6.0), (7.0, 8.0), (9.0, 10.0), 11.313708498984761)
((7.0, 8.0), (9.0, 10.0), (11.0, 12.0), 11.313708498984761)
```

```
[ ]:
```

26.

Mô tả: Đọc anscombe.csv, mỗi quý có x, y, z. Tính các giá trị thống kê cho z với từng quý. Tính dãy z_ standardized values cho z. Tính corr(y,z) cho quý I.

Code:

```
import csv
def row_iter(source):
    return csv.reader(source, delimiter="\t")
def head_split_fixed(row_iter):
    title= next(row_iter)
    assert len(title) == 1 and title[0] == "Anscombe's quartet"
    heading= next(row_iter)
    assert len(heading) == 4 and heading == ['I', 'II', 'III', 'IV']
    columns= next(row_iter)
    assert len(columns) == 12 and columns == ['x', 'y','z', 'x', 'y','z', 'x',
        'y','z', 'x', 'y','z']
    return row_iter
with open("Anscombe.csv") as source:
    print( list(head_split_fixed(row_iter(source))))
from collections import namedtuple
Trip = namedtuple("Trip", ("x", "y", "z"))
def series(n, row_iter):
    for row in row_iter:
        yield Trip(*row[n*3:n*3+3])
with open("Anscombe.csv") as source:
    data = tuple(head_split_fixed(row_iter(source)))
    sample_I= tuple(series(0,data))
    sample_II= tuple(series(1,data))
    sample_III= tuple(series(2,data))
    sample_IV= tuple(series(3,data))
mean = sum(float(trip.z) for trip in sample_I)/len(sample_I)
def s0(data):
    return sum(1 for x in data) # or len(data)
def s1(data):
    return sum(x for x in data) # or sum(data)
def s2(data):
```

```

    return sum(x*x for x in data)
def mean1(x):
    return s1(x)/s0(x)
def stdev1(x):
    return math.sqrt(s2(x)/s0(x) - (s1(x)/s0(x))**2)
def standard( x,  $\mu_x$ ,  $\sigma_x$  ):
    return (x- $\mu_x$ )/ $\sigma_x$ 
i = 1
for quy in sample_I,sample_II,sample_III,sample_IV:
    z_values = [float(trip.z) for trip in quy]
    mean_z, std_z = mean1(z_values), stdev1(z_values)
    z_standard = [round(standard(z,mean_z,std_z),3) for z in z_values]
    print(f"Mean(z) quý {i}: {mean_z:.3f}")
    print(f"Stdev(z) của quý {i}: {std_z:.3f}")
    print(f"danh sách các giá trị z sau khi chuẩn hóa quý {i}:", z_standard)
    print("-----")
    i +=1
def corr( sample1, sample2 ):
     $\mu_1$ ,  $\sigma_1$  = mean1(sample1), stdev1(sample1)
     $\mu_2$ ,  $\sigma_2$  = mean1(sample2), stdev1(sample2)
    z_1 = (standard(x,  $\mu_1$ ,  $\sigma_1$ ) for x in sample1)
    z_2 = (standard(x,  $\mu_2$ ,  $\sigma_2$ ) for x in sample2)
    r = sum(zx1*zx2 for zx1, zx2 in zip(z_1, z_2) )/s0(sample1)
    return r
y1_values,z1_values = [float(trip.y) for trip in sample_I], [float(trip.z) for trip in
sample_I]
sample_I
y1_values
z1_values
corr(y1_values,z1_values)

```

Giao diện:

```
[1]: import csv
def row_iter(source):
    return csv.reader(source, delimiter="\t")
```

```
[3]: def head_split_fixed(row_iter):
    title= next(row_iter)
    assert len(title) == 1 and title[0] == "Anscombe's quartet"
    heading= next(row_iter)
    assert len(heading) == 4 and heading == ['I', 'II', 'III', 'IV']
    columns= next(row_iter)
    assert len(columns) == 12 and columns == ['x', 'y', 'z', 'x', 'y', 'z', 'x',
    'y', 'z', 'x', 'y', 'z']
    return row_iter
```

```
[4]: with open("Anscombe.csv") as source:
    print( list(head_split_fixed(row_iter(source))))
```

```
[[ '10.0', '8.04', '21', '10.0', '9.14', '23.4', '10.0', '7.46', '3.4', '8.0', '6.58', '7.
7'], ['8.0', '6.95', '8.0', '8.14', '8.0', '6.77', '8.0', '5.76', '21', '5', '67', '8'],
['13.0', '7.58', '13.0', '8.74', '13.0', '12.74', '8.0', '7.71', '7', '9', '0', '-2.2']]
```

```
[6]: from collections import namedtuple
Trip = namedtuple("Trip", ("x", "y", "z"))
def series(n, row_iter):
    for row in row_iter:
        yield Trip(*row[n*3:n*3+3])
```

```
[8]: with open("Anscombe.csv") as source:
    data = tuple(head_split_fixed(row_iter(source)))
    sample_I= tuple(series(0,data))
    sample_II= tuple(series(1,data))
    sample_III= tuple(series(2,data))
    sample_IV= tuple(series(3,data))
```

```
[9]: mean = sum(float(trip.z) for trip in sample_I)/len(sample_I)
```

```
[15]: def s0(data):
    return sum(1 for x in data) # or len(data)
def s1(data):
    return sum(x for x in data) # or sum(data)
def s2(data):
    return sum(x*x for x in data)
```

```
[27]: def mean1(x):
    return s1(x)/s0(x)
def stdev1(x):
    return math.sqrt(s2(x)/s0(x) - (s1(x)/s0(x))**2)
```

```
[32]: def standard( x,  $\mu_x$ ,  $\sigma_x$  ):
    return (x- $\mu_x$ )/ $\sigma_x$ 
```

```
[43]: i = 1
      for quy in sample_I,sample_II,sample_III,sample_IV:
          z_values = [float(trip.z) for trip in quy]
          mean_z, std_z = mean1(z_values), stdev1(z_values)
          z_standard = [round(standard(z,mean_z,std_z),3) for z in z_values]
          print(f"Mean(z) quý {i}: {mean_z:.3f}")
          print(f"Stdev(z) của quý {i}: {std_z:.3f}")
          print(f"danh sách các giá trị z sau khi chuẩn hóa quý {i}:", z_standard)
          print("-----")
          i +=1
```

```
Mean(z) quý 1: 14.000
Stdev(z) của quý 1: 5.354
danh sách các giá trị z sau khi chuẩn hóa quý 1: [1.307, -1.121, -0.187]
-----
Mean(z) quý 2: 14.303
Stdev(z) của quý 2: 6.879
danh sách các giá trị z sau khi chuẩn hóa quý 2: [1.322, -1.095, -0.227]
-----
Mean(z) quý 3: 10.467
Stdev(z) của quý 3: 7.592
danh sách các giá trị z sau khi chuẩn hóa quý 3: [-0.931, 1.387, -0.457]
-----
Mean(z) quý 4: 4.500
Stdev(z) của quý 4: 4.739
danh sách các giá trị z sau khi chuẩn hóa quý 4: [0.675, 0.739, -1.414]
-----
```

Corr(y,z) cho quý I

```

[56]: def corr( sample1, sample2 ):
         $\mu_1$ ,  $\sigma_1$  = mean1(sample1), stdev1(sample1)
         $\mu_2$ ,  $\sigma_2$  = mean1(sample2), stdev1(sample2)
        z_1 = (standard(x,  $\mu_1$ ,  $\sigma_1$ ) for x in sample1)
        z_2 = (standard(x,  $\mu_2$ ,  $\sigma_2$ ) for x in sample2)
        r = sum(zx1*zx2 for zx1, zx2 in zip(z_1, z_2 ))/s0(sample1)
        return r

[49]: y1_values,z1_values = [float(trip.y) for trip in sample_I], [float(trip.z) for trip in samp
[50]: sample_I
[50]: (Trip(x='10.0', y='8.04', z='21'),
      Trip(x='8.0', y='6.95', z='8.0'),
      Trip(x='13.0', y='7.58', z='13.0'))
[51]: y1_values
[51]: [8.04, 6.95, 7.58]
[52]: z1_values
[52]: [21.0, 8.0, 13.0]
[57]: corr(y1_values,z1_values)
[57]: 0.9754022298277416
[ ]:

```

27

Mô tả: Dùng mẫu `unwrap(process(wrap))` để in ra chu vị bộ 4 lớn nhất và nhỏ nhất.

Code:

```

def read_points(filename):
    with open(filename) as f:
        for line in f:
            yield tuple(map(float, line.split()))
points = read_points("map.txt")
list(points)
def triplets(iterable):
    def triplet_from(first, second, iterable_tail):
        try:

```

```

        third = next(iterable_tail)
        yield first, second, third
        yield from triplet_from(second, third, iterable_tail)
    except StopIteration:
        return

    try:
        iterator = iter(iterable)
        return triplet_from(next(iterator), next(iterator), iterator) #
    except StopIteration:
        return

trips_iter = triplets(read_points("map.txt"))
for trip in trips_iter:
    print(trip)
import math

def chuvi(point1, point2, point3):
    def distance(p1, p2):
        return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)

    return distance(point1, point2) + distance(point2, point3) + distance(point3,
point1)
trips_iter = triplets(read_points("map.txt"))
quad =list(((triplet[0], triplet[1], triplet[2],chuvi(*triplet)) for triplet in trips_iter)
for quad_iter in quad:
    print(quad_iter)
# unwarp, process, wrap
def wrap(leg_iter):
    return ((leg[3],leg) for leg in leg_iter)
def unwrap(dist_leg):
    distance, leg = dist_leg
    return leg
long, short = unwrap(max(wrap(quad))), unwrap(min(wrap(quad)))
print(long, short)
Giao diện:

```

```
[12]: def read_points(filename):
      with open(filename) as f:
          for line in f:
              yield tuple(map(float, line.split()))
```

```
[13]: points = read_points("map.txt")
```

```
[14]: list(points)
```

```
[14]: [(1.0, 2.0),
      (5.0, 6.0),
      (9.0, 10.0),
      (11.0, 12.0),
      (8.5, 10.7),
      (9.6, 3.7),
      (3.0, 4.0),
      (8.0, 10.0)]
```

```
[15]: list(points)
```

```
[15]: []
```

```
[16]: def triplets(iterable):
      def triplet_from(first, second, iterable_tail):
          try:
              third = next(iterable_tail)
              yield first, second, third
              yield from triplet_from(second, third, iterable_tail)
          except StopIteration:
              return

      try:
          iterator = iter(iterable)
          return triplet_from(next(iterator), next(iterator), iterator) #
      except StopIteration:
          return
```

```
[17]: trips_iter = triplets(read_points("map.txt"))
```

```
[18]: for trip in trips_iter:
      print(trip)
```

```
((1.0, 2.0), (5.0, 6.0), (9.0, 10.0))
((5.0, 6.0), (9.0, 10.0), (11.0, 12.0))
((9.0, 10.0), (11.0, 12.0), (8.5, 10.7))
((11.0, 12.0), (8.5, 10.7), (9.6, 3.7))
((8.5, 10.7), (9.6, 3.7), (3.0, 4.0))
((9.6, 3.7), (3.0, 4.0), (8.0, 10.0))
```

```
[19]: import math

      def chuvi(point1, point2, point3):
          def distance(p1, p2):
              return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)

          return distance(point1, point2) + distance(point2, point3) + distance(point3, point1)
```

27

```
[73]: trips_iter = triplets(read_points("map.txt"))
      quad = list((triplet[0], triplet[1], triplet[2], chuvi(*triplet)) for triplet in trips_iter)
```

```
[74]: for quad_iter in quad:
      print(quad_iter)
```

```
((1.0, 2.0), (5.0, 6.0), (9.0, 10.0), 22.627416997969522)
((5.0, 6.0), (9.0, 10.0), (11.0, 12.0), 16.970562748477143)
((9.0, 10.0), (11.0, 12.0), (8.5, 10.7), 6.506460212171527)
((11.0, 12.0), (8.5, 10.7), (9.6, 3.7), 18.32094626055141)
((8.5, 10.7), (9.6, 3.7), (3.0, 4.0), 22.361049331994053)
((9.6, 3.7), (3.0, 4.0), (8.0, 10.0), 20.917064339570224)
```

```
[75]: def wrap(leg_iter):
      return ((leg[3], leg) for leg in leg_iter)
```

```
[76]: def unwrap(dist_leg):
      distance, leg = dist_leg
      return leg
```

```
[77]: long, short = unwrap(max(wrap(quad))), unwrap(min(wrap(quad)))
      print(long, short)
```

```
((1.0, 2.0), (5.0, 6.0), (9.0, 10.0), 22.627416997969522) ((9.0, 10.0), (11.0, 12.0), (8.5, 10.7), 6.506460212171527)
```

```
[ ]:
```

28.

Mô tả: Sắp xếp bộ 4 theo chu vi tăng dần với dữ liệu bài 27

Code:

```
quad_sorted = sorted(quad, key=lambda x: x[3])
```

```
for q in quad_sorted:
```

```
    print(q)
```

Giao diện:

```

28
[79]: quad
[79]: [(1.0, 2.0), (5.0, 6.0), (9.0, 10.0), 22.627416997969522),
      ((5.0, 6.0), (9.0, 10.0), (11.0, 12.0), 16.970562748477143),
      ((9.0, 10.0), (11.0, 12.0), (8.5, 10.7), 6.506460212171527),
      ((11.0, 12.0), (8.5, 10.7), (9.6, 3.7), 18.32094626055141),
      ((8.5, 10.7), (9.6, 3.7), (3.0, 4.0), 22.361049331994053),
      ((9.6, 3.7), (3.0, 4.0), (8.0, 10.0), 20.917064339570224)]
* [80]: quad_sorted = sorted(quad, key=lambda x: x[3])
      for q in quad_sorted:
          print(q)
      ((9.0, 10.0), (11.0, 12.0), (8.5, 10.7), 6.506460212171527)
      ((5.0, 6.0), (9.0, 10.0), (11.0, 12.0), 16.970562748477143)
      ((11.0, 12.0), (8.5, 10.7), (9.6, 3.7), 18.32094626055141)
      ((9.6, 3.7), (3.0, 4.0), (8.0, 10.0), 20.917064339570224)
      ((8.5, 10.7), (9.6, 3.7), (3.0, 4.0), 22.361049331994053)
      ((1.0, 2.0), (5.0, 6.0), (9.0, 10.0), 22.627416997969522)
[ ]:

```

29.

Mô tả: Tạo hàm hợp filter, function, calculation, reduce() dùng class. Kế thừa callable (tương tự sum__filter). Áp dụng: Nhân đôi các số chẵn và tính tổng trong một list.

Code:

```
from collections.abc import Callable
```

```
from functools import reduce
```

```
class Reduce_Filter(Callable):
```

```
    __slots__ = ["filter", "function", "calculation"]
```

```
    def __init__(self, filter, function, calculation):
```

```
        self.filter = filter # Bộ lọc chọn phần tử hợp lệ
```

```
        self.function = function # Hàm biến đổi giá trị
```

```
        self.calculation = calculation # Hàm tính toán reduce
```

```
    def __call__(self, iterable):
```

```

        filtered_transformed = [self.function(x) for x in iterable if self.filter(x)]
        return reduce(self.calculation, filtered_transformed) if filtered_transformed else
None

```

Ví dụ sử dụng:

```
is_even = lambda x: x % 2 == 0 # Bộ lọc: chọn số chẵn
```

```
double = lambda x: x * 2 # Biến đổi: nhân đôi
```

```
sum_values = lambda x, y: x + y # Tính toán: cộng dồn
```

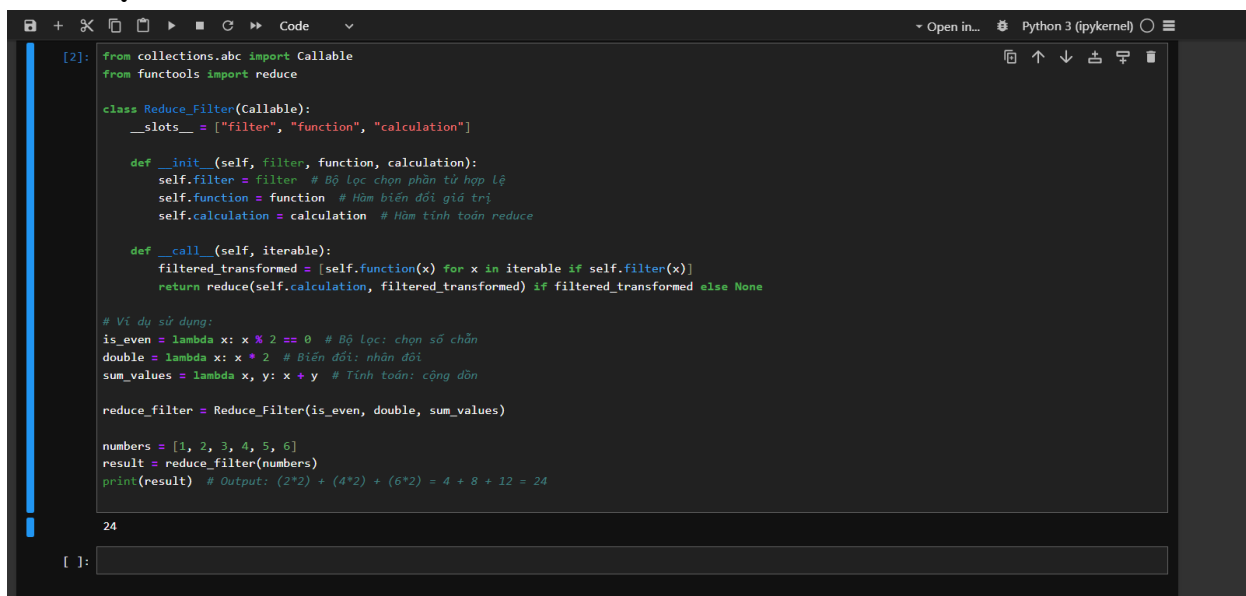
```
reduce_filter = Reduce_Filter(is_even, double, sum_values)
```

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
result = reduce_filter(numbers)
```

```
print(result) # Output: (2*2) + (4*2) + (6*2) = 4 + 8 + 12 = 24
```

Giao diện:



```

[2]: from collections.abc import Callable
from functools import reduce

class Reduce_Filter(Callable):
    __slots__ = ["filter", "function", "calculation"]

    def __init__(self, filter, function, calculation):
        self.filter = filter # Bộ lọc: chọn phần tử hợp lệ
        self.function = function # Hàm biến đổi giá trị
        self.calculation = calculation # Hàm tính toán reduce

    def __call__(self, iterable):
        filtered_transformed = [self.function(x) for x in iterable if self.filter(x)]
        return reduce(self.calculation, filtered_transformed) if filtered_transformed else None

# Ví dụ sử dụng:
is_even = lambda x: x % 2 == 0 # Bộ lọc: chọn số chẵn
double = lambda x: x * 2 # Biến đổi: nhân đôi
sum_values = lambda x, y: x + y # Tính toán: cộng dồn

reduce_filter = Reduce_Filter(is_even, double, sum_values)

numbers = [1, 2, 3, 4, 5, 6]
result = reduce_filter(numbers)
print(result) # Output: (2*2) + (4*2) + (6*2) = 4 + 8 + 12 = 24

24
[ ]:

```

30

Mô tả: quantized (bộ 4 có chu vi làm tròn đến 0,5,10,15,...). Chạy hàm đếm (counter): chu vi: số bộ đếm cùng chu vi.

Code:

```
def read_points(filename):
```

```
    with open(filename) as f:
```

```
        for line in f:
```

```

        yield tuple(map(float, line.split()))
def triplets(iterable):
    def triplet_from(first, second, iterable_tail):
        try:
            third = next(iterable_tail)
            yield first, second, third
            yield from triplet_from(second, third, iterable_tail)
        except StopIteration:
            return

    try:
        iterator = iter(iterable)
        return triplet_from(next(iterator), next(iterator), iterator) #
    except StopIteration:
        return

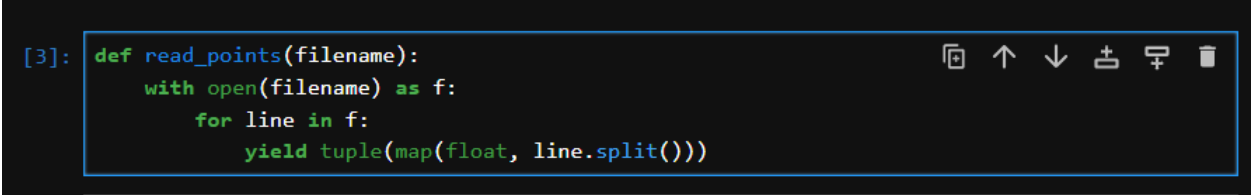
import math

def chuvi(point1, point2, point3):
    def distance(p1, p2):
        return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)

    return distance(point1, point2) + distance(point2, point3) + distance(point3,
point1)
trips_iter = triplets(read_points("map.txt"))
quad =list(((triplet[0], triplet[1], triplet[2], chuvi(*triplet)) for triplet in trips_iter)
quad
quantized= (5*(chu_vi//5) for p1,p2,p3, chu_vi in quad)
list(quantized)
from collections import Counter
Counter((5*(chu_vi//5) for p1,p2,p3, chu_vi in quad))

```

Giao diện:



```

[3]: def read_points(filename):
      with open(filename) as f:
          for line in f:
              yield tuple(map(float, line.split()))

```

```
[2]: def triplets(iterable):
      def triplet_from(first, second, iterable_tail):
          try:
              third = next(iterable_tail)
              yield first, second, third
              yield from triplet_from(second, third, iterable_tail)
          except StopIteration:
              return

      try:
          iterator = iter(iterable)
          return triplet_from(next(iterator), next(iterator), iterator) #
      except StopIteration:
          return
```

```
[5]: import math

      def chuvi(point1, point2, point3):
          def distance(p1, p2):
              return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)

          return distance(point1, point2) + distance(point2, point3) + distance(point3, point1)
```

```
[6]: trips_iter = triplets(read_points("map.txt"))
      quad = list((triplet[0], triplet[1], triplet[2], chuvi(*triplet)) for triplet in trips_iter)
```

```
[7]: for quad_iter in quad:
```

```
[7]: quad
```

```
[7]: (((1.0, 2.0), (5.0, 6.0), (9.0, 10.0), 22.627416997969522),
      ((5.0, 6.0), (9.0, 10.0), (11.0, 12.0), 16.970562748477143),
      ((9.0, 10.0), (11.0, 12.0), (8.5, 10.7), 6.506460212171527),
      ((11.0, 12.0), (8.5, 10.7), (9.6, 3.7), 18.32094626055141),
      ((8.5, 10.7), (9.6, 3.7), (3.0, 4.0), 22.361049331994053),
      ((9.6, 3.7), (3.0, 4.0), (8.0, 10.0), 20.917064339570224])
```

```
[12]: quantized = (5*(chu_vi//5) for p1,p2,p3, chu_vi in quad)
```

```
[13]: list(quantized)
```

```
[13]: [20.0, 15.0, 5.0, 15.0, 20.0, 20.0]
```

```
[16]: list(quantized)
```

```
[16]: []
```

```
[15]: from collections import Counter
      Counter((5*(chu_vi//5) for p1,p2,p3, chu_vi in quad))
```

```
[15]: Counter({20.0: 3, 15.0: 2, 5.0: 1})
```

```
[ ]:
```

31

Mô tả: In ra chu vi và danh sách các bộ đếm co cùng chu vi (chu vi đã lượng tử hóa bởi số gần nhất của 5). Tạo từ điển.

Code:

```
quad
from collections import defaultdict
def group_by(key, data):
    def group_into(key, collection, dictionary):
        if len(collection) == 0:
            return dictionary
        head, *tail= collection
        dictionary[key(head)].append(head)
        return group_into(key, tail, dictionary)
    return group_into(key, data, defaultdict(list))
chuvi = lambda last: 5*(last[3]//5)
by_chuvi = group_by(chuvi,quad)
import pprint
for chuvi in sorted(by_chuvi):
    print(chuvi)
    pprint.pprint(by_chuvi[chuvi])
```

Giao diện:

```
[17]: quad

[17]: (((1.0, 2.0), (5.0, 6.0), (9.0, 10.0), 22.627416997969522),
      ((5.0, 6.0), (9.0, 10.0), (11.0, 12.0), 16.970562748477143),
      ((9.0, 10.0), (11.0, 12.0), (8.5, 10.7), 6.506460212171527),
      ((11.0, 12.0), (8.5, 10.7), (9.6, 3.7), 18.32094626055141),
      ((8.5, 10.7), (9.6, 3.7), (3.0, 4.0), 22.361049331994053),
      ((9.6, 3.7), (3.0, 4.0), (8.0, 10.0), 20.917064339570224)]

[20]: from collections import defaultdict

[15]: def group_by(key, data):
      def group_into(key, collection, dictionary):
          if len(collection) == 0:
              return dictionary
          head, *tail = collection
          dictionary[key(head)].append(head)
          return group_into(key, tail, dictionary)
      return group_into(key, data, defaultdict(list))

[16]: chuvi = lambda last: 5*(last[3]//5)

[21]: by_chuvi = group_by(chuvi, quad)

[23]: import pprint
      for chuvi in sorted(by_chuvi):
          print(chuvi)
          pprint.pprint(by_chuvi[chuvi])

5.0
[((9.0, 10.0), (11.0, 12.0), (8.5, 10.7), 6.506460212171527)]
15.0
[((5.0, 6.0), (9.0, 10.0), (11.0, 12.0), 16.970562748477143),
 ((11.0, 12.0), (8.5, 10.7), (9.6, 3.7), 18.32094626055141)]
20.0
[((1.0, 2.0), (5.0, 6.0), (9.0, 10.0), 22.627416997969522),
 ((8.5, 10.7), (9.6, 3.7), (3.0, 4.0), 22.361049331994053),
 ((9.6, 3.7), (3.0, 4.0), (8.0, 10.0), 20.917064339570224)]

[ ]:
```

32.

Mô tả: Với mỗi chu vi từ bé đến lớn, in ra bộ 4 mà điểm giữa có tọa độ long là lớn nhất. Dùng dữ liệu đã được lượng tử hóa bài 31 (by_chuvi).

Code:

```
import pprint
for chuvi in sorted(by_chuvi):
    max_long_mid = max(by_chuvi[chuvi], key=lambda x: x[1][0])
    print(chuvi)
```

```
pprint.pprint(max_long_mid)
```

Giao diện:

```
[14]: import pprint
      for chuvi in sorted(by_chuvi):
          max_long_mid = max(by_chuvi[chuvi], key=lambda x: x[1][0])
          print(chuvi)
          pprint.pprint(max_long_mid)
```

```
5.0
((9.0, 10.0), (11.0, 12.0), (8.5, 10.7), 6.506460212171527)
15.0
((5.0, 6.0), (9.0, 10.0), (11.0, 12.0), 16.970562748477143)
20.0
((8.5, 10.7), (9.6, 3.7), (3.0, 4.0), 22.361049331994053)
```

```
[ ]:
```